

**Escuela Técnica Superior de Ingeniería
Universidad de Huelva**

Grado en Ingeniería Informática

Trabajo Fin de Grado

Predicción y explicación del éxito en proyectos de
crowdfunding mediante modelos de lenguaje y
métodos de interpretabilidad

Autor: Daniel Dorado Martínez

Tutor: Jacinto Mata Vázquez

Tutora: Victoria Pachón Álvarez

Junio, 2025



Este Trabajo Fin de Grado se enmarca en una Beca de Colaboración concedida a su autor por el Ministerio de Educación, Formación Profesional y Deportes de España en la convocatoria 2024/2025, vinculada al Grupo de Investigación I2C (Ingeniería de la Información y del Conocimiento) de la Universidad de Huelva.

Para Arantxa

*A ti, que creíste en mí incluso cuando yo dudaba.
Que me enseñaste que un poco de locura no solo es necesaria,
sino que, como diría el Sombrero, las mejores personas la llevan consigo.
Que los sueños pueden enredarse pero, también, hacerse realidad.
Eres mi segunda estrella a la derecha.
Gracias por estar en esta aventura.
Y por no dudar en lanzarte a la siguiente conmigo.*

Resumen

Este trabajo presenta el desarrollo de un sistema automatizado para la predicción del éxito de proyectos de *crowdfunding* en la plataforma Goteo a partir de los textos redactados por sus promotores. La propuesta se basa en un meta-clasificador que integra modelos de lenguaje preentrenados en español, ajustados individualmente sobre cuatro campos textuales: *about*, *goal*, *motivation* y *description*. Para afrontar problemas como el desbalance de clases, la longitud de los textos y su variabilidad semántica, se adoptan estrategias de segmentación por campo, combinación mediante técnicas de *ensemble* y entrenamiento con *underbagging*. La búsqueda de hiperparámetros se realiza con herramientas especializadas para dicha tarea. Los resultados muestran un mejor rendimiento en uno de los modelos, que presenta una precisión del 76,22 % y un AUC de 0,6759 sobre un conjunto de prueba no balanceado, siendo el campo *goal* el más informativo. Además, se incorpora un análisis de interpretabilidad con *Integrated Gradients* a fin de identificar los *tokens* más influyentes en la decisión del modelo. El proyecto, que se enfoca exclusivamente en el análisis del lenguaje empleado por los usuarios, introduce futuras líneas de actuación en la mejora de sistemas de apoyo a la redacción de campañas de financiación colectiva con orientación social.

Palabras clave: aprendizaje automático, procesamiento del lenguaje natural, Transformers, *crowdfunding*, clasificación binaria, interpretabilidad

Abstract

This study develops an automated system to predict the success of crowdfunding projects on the Goteo platform using the textual content provided by project creators. The approach relies on a meta-classifier that integrates Spanish pre-trained language models, each fine-tuned on four different textual fields: *about*, *goal*, *motivation*, and *description*. To address challenges such as class imbalance, text length, and semantic variability, the method applies field-wise segmentation, ensemble techniques, and underbagging-based training. Hyperparameter optimization uses tools specifically designed for this purpose. One model achieves the highest performance, reaching an accuracy of 76.22 % and an AUC of 0.6759 on an imbalanced test set, with the *goal* field providing the most informative input. The study also incorporates an interpretability analysis using *Integrated Gradients* to identify the most influential *tokens* in the model’s decision-making process. By focusing exclusively on the language used by creators, this project outlines future directions for enhancing support systems aimed at drafting socially-oriented crowdfunding campaigns.

Keywords: machine learning, natural language processing, Transformers, crowdfunding, binary classification, interpretability

Agradecimientos

Quiero expresar mi más sincero agradecimiento a mis tutores por su paciencia y comprensión. Durante el desarrollo de este trabajo he atravesado momentos de duda y falta de claridad, lo que en ocasiones me llevó a dar giros en el enfoque. Aun así, siempre estuvieron disponibles para orientarme, resolver mis inquietudes y animarme a continuar con confianza. Su acompañamiento ha sido esencial para la finalización de este trabajo.

También quiero dedicar unas palabras muy especiales a Runa y Terra, mis perritas, que han sido un apoyo silencioso, pero constante, a lo largo de este proceso. Su compañía, su cariño incondicional y su forma de estar sin pedir nada a cambio, han sido un apoyo en los días más complicados y una alegría en momentos de descanso. Gracias por recordarme, con solo una mirada o, especialmente, con una cola moviéndose, que todo va a estar bien.

No puedo olvidar a mis compañeros y compañeras de clase, con quienes he compartido apuntes, trabajos y, cómo no, la preocupación constante por los exámenes. Pero también hemos compartido muchas risas y anécdotas que han hecho este camino mucho más llevadero. Gracias por los momentos absurdos, los chistes a deshoras y por demostrar que, incluso en medio del agobio, siempre hay espacio para el buen humor.

También deseo dar las gracias a los docentes de la titulación, quienes han contribuido en mi crecimiento académico y personal. Aunque algunas asignaturas me hayan hecho sentir que pierdo años de vida, cada clase, cada explicación y cada comentario han dejado huella y han formado parte de este aprendizaje.

A mi familia, por estar siempre ahí y por su apoyo. Aunque no sepan muy bien en qué consiste este trabajo (siquiera estudios), su confianza ha sido importante para seguir adelante.

Daniel Dorado Martínez
Huelva, 2025

Índice general

Resumen	III
Abstract	IV
Agradecimientos	V
Índice de Figuras	IX
Índice de Tablas	X
1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Competencias	3
1.4. Estructura de la memoria	4
2. Marco teórico	5
2.1. Aprendizaje automático	5
2.1.1. Aprendizaje supervisado	5
2.1.2. Aprendizaje no supervisado	6
2.1.3. Aprendizaje por refuerzo	6
2.2. Redes neuronales artificiales	6
2.2.1. Neurona artificial y perceptrón	7
2.2.2. Arquitectura de una red neuronal	7
2.2.3. Propagación hacia adelante (<i>forward propagation</i>)	7
2.2.4. Retropropagación y función de activación	7
2.2.5. Limitaciones de las redes tradicionales	8
2.3. Aprendizaje profundo	8
2.3.1. <i>Epochs</i> y tamaño de lote	8
2.3.2. Tasa de aprendizaje y <i>weight decay</i>	9
2.3.3. Función de pérdida	9
2.3.4. <i>Overfitting</i> y técnicas de regularización	9
2.3.5. <i>Early stopping</i> y validación	9
2.4. Procesamiento del lenguaje natural (PLN)	10
2.4.1. Preprocesamiento del texto	10
2.4.2. Representación del texto	10
2.4.3. Clasificación de texto en PLN	11
2.5. Transformers: fundamentos, estructura y aplicaciones	11
2.5.1. Introducción	11
2.5.2. Contexto: limitaciones de las arquitecturas previas	12
2.5.3. Mecanismo de <i>self-attention</i>	12
2.5.4. <i>Word embeddings</i> : representación semántica del lenguaje	12
2.5.5. <i>Positional encoding</i>	12
2.5.6. Arquitectura general del transformer	13
2.5.7. Limitaciones y retos actuales	14

2.6.	Modelos seleccionados: BETO y RoBERTa-base-bne	14
2.6.1.	RoBERTa: <i>Robustly optimized BERT pretraining approach</i>	14
2.6.2.	RoBERTa-base-bne	15
2.6.3.	BETO: BERT para el español	15
2.7.	Aprendizaje por transferencia	15
2.7.1.	<i>Fine-tuning</i>	16
2.8.	<i>Ensemble</i> de modelos	16
2.8.1.	<i>Ensemble</i> simple	17
2.8.2.	Tipos de <i>ensemble</i>	17
2.8.3.	<i>Ensembles</i> con modelos de lenguaje	18
2.9.	Interpretabilidad de modelos de AA	18
2.9.1.	<i>Integrated Gradients</i>	19
2.9.2.	Interpretabilidad con Captum	20
2.10.	Medidas de evaluación	20
2.10.1.	Matriz de confusión	21
2.10.2.	Precisión	21
2.10.3.	Sensibilidad o recobrado	21
2.10.4.	Medida F1	21
2.10.5.	Exactitud	22
2.10.6.	Curva ROC y AUC	22
2.11.	Trabajos relacionados	22
2.12.	Tecnologías y recursos utilizados	23
3.	Metodología, experimentación y resultados	26
3.1.	Descripción general de la metodología	26
3.2.	Extracción de los datos	27
3.3.	Descripción del conjunto de datos	28
3.4.	Preparación de los datos	29
3.4.1.	Selección de campos relevantes	29
3.4.2.	Limpieza y normalización	30
3.4.3.	Detección de idioma	30
3.4.4.	Traducción automática al español	31
3.4.5.	<i>Tokenización</i> y análisis de longitud	31
3.5.	Balanceo del <i>dataset</i>	32
3.5.1.	Consideraciones sobre técnicas de balanceo	32
3.5.2.	Evaluación de proporciones de clases	33
3.5.3.	Resultados comparativos	33
3.5.4.	División del conjunto de datos	34
3.6.	Modelado y entrenamiento	34
3.6.1.	Búsqueda de hiperparámetros con Optuna	35
3.6.2.	<i>Underbagging</i> y generación de modelos	36
3.6.3.	Resultados del <i>ensemble</i> por campo y comparativa de rendimiento	37
3.6.4.	Resultados finales	38
3.6.5.	Interpretabilidad del modelo	41
3.6.6.	Contribución del modelo en predicción de proyectos <i>crowdfunding</i>	53
3.6.7.	Limitaciones	54

4. Conclusiones y trabajo futuro	56
4.1. Conclusiones	56
4.2. Trabajo futuro	58
4.2.1. Mejoras técnicas y evolución arquitectónica	58
4.2.2. Validación externa y generalización	59
4.2.3. Aplicaciones prácticas y transferencia de conocimiento	59
4.3. Planificación temporal del trabajo realizado	60
Referencias	61
A. Anexos	65
A.1. Tablas de resultado de búsqueda hiperparámetros con Optuna	65
A.2. Repositorio de código utilizado	67

Índice de Figuras

2.1. Arquitectura Transformer.	13
2.2. Tecnologías y recursos utilizados en el desarrollo del trabajo.	24
3.1. <i>Pipeline</i> del marco de experimentación.	27
3.2. Esquema del sistema de clasificación jerárquico con meta-clasificador.	38
3.3. Matriz de confusión del modelo final BETO.	39
3.4. Matriz de confusión del modelo final RoBERTa.	39
3.5. Curva ROC del modelo final BETO.	40
3.6. Curva ROC del modelo final RoBERTa.	40
3.7. <i>Tokens</i> más influyentes para la predicción en el campo about (BETO).	42
3.8. <i>Tokens</i> más influyentes para la predicción en el campo description (BETO).	43
3.9. <i>Tokens</i> más influyentes para la predicción en el campo motivation (BETO).	44
3.10. <i>Tokens</i> más influyentes para la predicción en el campo goal (BETO).	45
3.11. <i>Tokens</i> más influyentes a nivel global (modelo BETO con campos fusionados).	46
3.12. <i>Tokens</i> más influyentes para la predicción en el campo about (RoBERTa).	47
3.13. <i>Tokens</i> más influyentes para la predicción en el campo description (RoBERTa).	48
3.14. <i>Tokens</i> más influyentes para la predicción en el campo motivation (RoBERTa).	49
3.15. <i>Tokens</i> más influyentes para la predicción en el campo goal (RoBERTa).	50
3.16. <i>Tokens</i> más influyentes a nivel global (modelo RoBERTa con campos fusionados).	51
3.17. Ejemplo de visualización de atribuciones por <i>token</i> mediante <i>Integrated Gradients</i>	52

Índice de Tablas

3.1. Campos principales del <i>dataset</i>	28
3.2. Distribución original de idiomas según el campo <i>lang</i>	30
3.3. Distribución de idiomas detectada con fastText.	31
3.4. Promedio de tokens por campo textual tras la tokenización.	32
3.5. Hiperparámetros utilizados durante la experimentación con proporciones.	33
3.6. Resultados del <i>ensemble</i> por votación mayoritaria según proporción de clases.	34
3.7. Distribución de ejemplos por conjunto tras división y balanceo (valores estimados).	34
3.8. Espacio de búsqueda de hiperparámetros con Optuna.	35
3.9. Hiperparámetros óptimos seleccionados por Optuna.	35
3.10. Resultados individuales por <i>bootstrap</i> , campo y arquitectura.	36
3.11. Resultados del <i>ensemble</i> por campo tras fusión de <i>bootstraps</i>	37
3.12. Resultados finales tras <i>ensemble</i> completo por arquitectura.	38
A.1. Tabla 1: Métricas e Hiperparámetros BETO <i>about</i>	65
A.2. Tabla 2: Métricas y Hiperparámetros BETO <i>description</i>	65
A.3. Tabla 3: Métricas y Hiperparámetros BETO <i>motivation</i>).	66
A.4. Tabla 4: Métricas y Hiperparámetros BETO <i>goal</i>	66
A.5. Tabla 5: Métricas y Hiperparámetros RoBERTa-base-bne <i>about</i>	66
A.6. Tabla 6: Métricas y Hiperparámetros RoBERTa-base-bne <i>description</i>	66
A.7. Tabla 7: Métricas y Hiperparámetros RoBERTa-base-bne <i>motivation</i>	67
A.8. Tabla 8: Métricas y Hiperparámetros RoBERTa-base-bne <i>goal</i>	67

Introducción

Este capítulo presenta una visión general del trabajo, ofreciendo el contexto, objetivos, competencias adquiridas y estructura de la memoria. Se introduce un proyecto orientado al diseño e implementación de un sistema de clasificación automática del éxito de proyectos de *crowdfunding*, empleando técnicas de aprendizaje automático y modelos de lenguaje preentrenados. Asimismo, se abordan retos asociados, como el desbalanceo de clases, la variabilidad del lenguaje en los textos y la necesidad de interpretabilidad sobre las decisiones del modelo.

1.1. Motivación

El auge del *crowdfunding* como modelo alternativo y orgánico de financiación ha transformado la forma en que emprendedores, colectivos sociales y organizaciones culturales presentan sus ideas ante una comunidad digital y abierta de posibles mecenases. A medida que crece el número de campañas y plataformas disponibles, también lo hace la competencia por captar la atención y el apoyo económico, generando un entorno donde la predicción del éxito de un proyecto adquiere un valor estratégico importante.

La posibilidad de anticipar el rendimiento de una campaña antes de su publicación no solo puede beneficiar a los promotores al facilitarles una validación temprana de sus contenidos, sino que puede resultar útil para las propias plataformas de *crowdfunding*, orientando su estrategia de negocio a los recursos, asesoramiento o visibilidad en función del análisis previo del texto. En este contexto, los modelos de aprendizaje automático aplicados al lenguaje natural emergen como herramientas prometedoras para detectar patrones predictivos, ofrecer interpretaciones basadas en datos y asistir en la toma de decisiones en estos entornos de alto volumen de información y competitividad.

Si bien, este tipo de predicción plantea retos. Las descripciones de campaña suelen estar redactadas en lenguaje libre, con alta variabilidad léxica, sintáctica y estilística. El lenguaje es por naturaleza ambiguo, subjetivo y dependiente del contexto, lo que hace que su modelización automática requiera arquitecturas complejas y una comprensión semántica profunda. A esta dificultad se añade el problema del

desbalanceo entre clases ya que, en la plataforma analizada, un número notablemente mayor de campañas alcanza su objetivo de financiación, mientras que solo una fracción minoritaria fracasa. Esta distribución desigual limita la capacidad de los clasificadores para generalizar correctamente sobre la clase menos representada y obliga a diseñar estrategias específicas de entrenamiento y validación que mitiguen estos efectos.

Así, la presente investigación se desarrolla tomando como base la plataforma Goteo¹. Un entorno que, en el contexto digital del *crowdfunding*, destaca por su explícita orientación hacia proyectos de cariz social, educativo, ambiental y/o comunitario. Goteo permite a sus usuarios una redacción discursiva libre con énfasis en valores socio-colectivos que buscan el beneficio público y la sostenibilidad. Esta característica amplía la riqueza semántica de las campañas, al tiempo que introduce una mayor variabilidad textual, lo que representa un desafío para los sistemas de predicción automatizada.

Por consiguiente, desde la óptica de la ingeniería informática, trabajar con este tipo de plataformas plantea un reto asociado al análisis de datos no estructurados, donde confluyen técnicas de procesamiento de lenguaje natural (PLN), aprendizaje profundo, interpretación de modelos y sistemas de decisión. De modo que, más allá de su relevancia estratégica, este tipo de experimentos pueden llegar a tener un impacto directo en la democratización de la financiación colectiva y en la generación de herramientas tecnológicas al servicio de la transformación social. Es decir, el análisis predictivo de este tipo de campañas representa una necesidad técnica y una oportunidad para sostener soluciones adaptadas a contextos de alta sensibilidad social y complejidad narrativa.

En este orden, el presente trabajo parte de la confluencia entre técnicas de PLN en lengua española en un dominio socio-narrativo de exploración emergente, combinando precisión técnica con sensibilidad contextual. Al centrarse exclusivamente en los textos redactados por los promotores sin utilizar variables estructuradas tradicionales (como, por ejemplo, el presupuesto, duración o número de recompensas), se propone aislar el lenguaje como variable explicativa para evaluar su potencial predictivo desde una perspectiva computacional, interpretativa y práctica.

1.2. Objetivos

El objetivo general de este trabajo fue desarrollar un sistema de clasificación que, a partir de los textos de proyectos de *crowdfunding* en la plataforma Goteo, adquiriera la capacidad para predecir su éxito o fracaso. Para alcanzarlo, se establecieron los

¹<https://www.goteo.org>

siguientes objetivos específicos:

- Estudiar las técnicas actuales de clasificación de texto basadas en aprendizaje profundo y modelos preentrenados.
- Seleccionar arquitecturas adecuadas para el procesamiento de textos en español.
- Implementar un sistema de entrenamiento mediante *underbagging* para abordar el desbalanceo de clases.
- Realizar una búsqueda óptima de hiperparámetros mediante para mejorar el rendimiento.
- Aplicar técnicas de *ensemble* para combinar predicciones a distintos niveles (por campo y global).
- Evaluar el sistema mediante métricas estándar como *accuracy*, F1 y AUC.
- Incorporar interpretabilidad en los modelos entrenados.

1.3. Competencias

Las competencias curriculares adquiridas en este trabajo han sido:

- **CE7-C - Capacidad para conocer y desarrollar técnicas de aprendizaje computacional y diseñar e implementar aplicaciones y sistemas que las utilicen.** Esta competencia se ha desarrollado a través del diseño, implementación y evaluación de un sistema completo de clasificación de texto basado en deep learning.

Además, y de acuerdo al plan de estudios, se ha desarrollado la:

- **Capacidad para aplicar técnicas de procesamiento del lenguaje natural.** Se han utilizado modelos de lenguaje preentrenados y técnicas de pre-procesamiento textual adaptadas al español.
- **Capacidad para la extracción automática de información.** Se ha trabajado con representaciones vectoriales del texto para extraer información relevante del lenguaje natural.
- **Capacidad para diseñar experimentos y evaluar resultados.** Se han aplicado métricas de evaluación estándar y se ha analizado la efectividad de los modelos en condiciones reales.
- **Capacidad para adquirir conocimiento a partir de grandes volúmenes de datos.** Se ha trabajado con un *dataset* textual de alta variabilidad, desarrollando mecanismos para su procesamiento y análisis.

1.4. Estructura de la memoria

La presente memoria se organiza de la siguiente manera:

- En el [Capítulo 2](#), Marco teórico, se describen los fundamentos del aprendizaje automático, redes neuronales, modelos Transformer, aprendizaje por transferencia, técnicas de ensemble y herramientas de interpretabilidad que sustentan este trabajo.
- En el [Capítulo 3](#), Metodología, experimentación y resultados, se expone el diseño del sistema, el procesamiento de los datos, el entrenamiento de modelos y las evaluaciones realizadas, así como los resultados obtenidos.
- En el [Capítulo 4](#) se presentan las conclusiones del trabajo, valorando su impacto y proponiendo futuras líneas de actuación, como el uso de LLMs o técnicas de ensamblado más sofisticadas.
- En el [A.1](#), se recogen las tablas con los resultados completos del proceso de búsqueda de hiperparámetros llevado a cabo con Optuna, ordenados por métrica y configuración.
- En el [A.2](#), se facilita el enlace al repositorio de GitHub que contiene todo el código desarrollado durante el trabajo, incluyendo procesamiento de datos, entrenamiento de modelos, análisis de interpretabilidad y la aplicación de demostración.

Marco teórico

2.1. Aprendizaje automático

El Aprendizaje Automático (AA) es una subdisciplina de la inteligencia artificial, orientada al diseño de algoritmos con la capacidad de identificar patrones en los datos para mejorar su rendimiento sin intervención humana explícita. Su objetivo es dotar a los sistemas informáticos de la capacidad de aprender de la experiencia para tomar decisiones o realizar predicciones de forma autónoma [1].

Este proceso se sustenta en la generalización del modelo a partir de datos previamente observados y ajusta su comportamiento conforme recibe nueva información. Esto permite automatizar tareas complejas y desarrollar sistemas adaptativos, fundamentales en entornos dinámicos [2]. Un ejemplo de esto es el uso de AA en vehículos autónomos, donde el sistema debe adaptarse en tiempo real a cambios constantes en el entorno, como condiciones climáticas, comportamiento impredecible de peatones o modificaciones en la señalización vial.

2.1.1. Aprendizaje supervisado

El aprendizaje supervisado ha resultado el marco principal de este trabajo. El procedimiento consiste en partir de un conjunto de datos etiquetados, donde cada instancia está representada por un vector de características de entrada asociado a una salida conocida. Durante el entrenamiento, el modelo ajusta sus parámetros internos con el objetivo de aproximar una función que relacione las entradas con las salidas, minimizando una función de pérdida definida sobre el conjunto del entrenamiento. Esta función aprendida permite realizar predicciones sobre nuevas instancias no previstas, en la medida en que logre capturar patrones en los datos [1].

Las dos tareas que delimitan el aprendizaje supervisado son la:

- Clasificación: asignación de instancias a clases o categorías discretas. Por ejemplo, identificar si un mensaje es *spam*.
- Regresión: predicción de valores continuos, como el precio de una vivienda en función de sus características.

El desempeño de estos métodos depende de la calidad, representatividad y pre-procesamiento del conjunto de datos. Esto resulta especialmente relevante en tareas de análisis textual, como la que compete en este trabajo, donde la representación semántica de los textos condiciona significativamente el rendimiento del modelo [2].

2.1.2. Aprendizaje no supervisado

En contraste, el aprendizaje no supervisado no dispone de etiquetas asociadas a los datos. La finalidad de este sistema es descubrir estructuras latentes o patrones ocultos que permitan describir o resumir el conjunto [1]. Las técnicas más comunes son:

- *Clustering*: agrupamiento de instancias similares. Es útil para la segmentación de clientes, identificación de temas en texto, entre otros.
- Reducción de dimensionalidad: métodos como el Análisis de Componentes Principales (PCA) facilitan la visualización de datos de alta dimensión y reducen la redundancia informativa.

2.1.3. Aprendizaje por refuerzo

Por último, el aprendizaje por refuerzo se inspira en la toma de decisiones secuenciales. Un agente aprende a través de la interacción con un entorno, recibiendo recompensas o penalizaciones en función de sus acciones. El objetivo es maximizar la recompensa acumulada a largo plazo. Este tipo de aprendizaje ha sido particularmente exitoso en dominios relacionados con los juegos, la robótica o la navegación autónoma [1].

En este proyecto se ha empleado el enfoque de aprendizaje supervisado, ya que la tarea consiste en un problema de clasificación binaria de proyectos de *crowdfunding*, donde cada instancia está etiquetada como exitosa o no exitosa.

2.2. Redes neuronales artificiales

Las Redes Neuronales Artificiales (RNA) son modelos computacionales inspirados en el cerebro humano, diseñados para aprender representaciones complejas a partir de datos. Se estructuran en capas formadas por *neuronas artificiales* que se conectan entre sí mediante pesos ajustables. Su flexibilidad para modelar relaciones no lineales las convierte en una herramienta esencial para tareas de clasificación, regresión o generación de datos, incluyendo el PLN, siendo el área focal de este trabajo.

2.2.1. Neurona artificial y perceptrón

La *neurona artificial*, propuesta inicialmente por McCulloch y Pitts en 1943, fue formalizada como modelo computacional por Rosenblatt con el *perceptrón* en 1958 [3]. Este modelo realiza una combinación lineal de las entradas, seguida de una función de activación binaria. Si bien resultaba útil para problemas linealmente separables, su dificultad para resolver tareas más complejas fue señalada por Minsky y Papert en 1969 [4], lo que impulsó la evolución hacia redes más profundas y no lineales.

2.2.2. Arquitectura de una red neuronal

Una RNA se organiza en tres tipos de capas: de entrada, ocultas y de salida. Las redes *feedforward* de múltiples capas, conocidas como *Perceptrones Multicapa* (MLP), han demostrado ser universales para la aproximación de funciones no lineales [5]. Estas arquitecturas son precursoras de modelos más sofisticados, como los Transformers empleados en este proyecto, que amplían significativamente la profundidad y la capacidad de representación semántica del lenguaje.

2.2.3. Propagación hacia adelante (*forward propagation*)

El mecanismo *forward propagation* permite que la información fluya desde las entradas hasta las salidas. En cada capa, las neuronas procesan una combinación lineal de los datos con los pesos y aplican una función de activación. Este principio sigue vigente en arquitecturas modernas, aunque en modelos como los Transformers se reemplazan las capas densas por bloques de atención y normalización más complejos [6].

2.2.4. Retropropagación y función de activación

El entrenamiento de redes se realiza mediante el algoritmo de *retropropagación* introducido por Rumelhart, Hinton y Williams en 1986 [7], que ajusta los pesos usando gradiente descendente. Las funciones de activación como *ReLU*, *sigmoide* o *tanh* son esenciales para introducir no linealidades, permitiendo a la red aprender representaciones complejas [8]. Aunque los Transformers utilizan funciones de activación similares, su entrenamiento optimiza bloques con estructuras especializadas como *self-attention*.

2.2.5. Limitaciones de las redes tradicionales

Ahora bien, las RNA tradicionales presentan limitaciones notables cuando se aplican a problemas complejos como el lenguaje natural. Dificultades como el desvanecimiento del gradiente dificultan el aprendizaje en redes profundas [9], y su necesidad de grandes volúmenes de datos y cómputo puede ser restrictiva.

Estas limitaciones motivaron el desarrollo de arquitecturas modernas como los Transformers, que superan muchas de estas barreras y han sido adoptadas en este trabajo como base para el modelado de texto en español.

2.3. Aprendizaje profundo

El aprendizaje profundo (*deep learning*) es una ramificación del aprendizaje automático que emplea redes neuronales con numerosas capas para extraer representaciones jerárquicas y no lineales de los datos. Su principal ventaja es la capacidad de aprender automáticamente características relevantes a partir de los datos, sin requerir una ingeniería manual intensiva [5].

En PLN, esta disciplina ha experimentado un gran avance gracias al desarrollo de arquitecturas como las redes neuronales recurrentes (RNN), las redes *Long Short-Term Memory* (LSTM) y, más recientemente, los modelos basados en atención, como los Transformers [10]. Estos últimos, por su capacidad para capturar relaciones contextuales de largo alcance y escalar de manera eficiente en paralelo, constituyen la base de los modelos empleados en este trabajo. De esta manera, el entrenamiento de modelos profundos implica la optimización de millones de parámetros, lo que requiere ajustar aspectos como la tasa de aprendizaje, el tamaño del lote y técnicas de regularización para garantizar la generalización y evitar el sobreajuste.

2.3.1. *Epochs* y tamaño de lote

Una *epoch* representa un recorrido completo por el conjunto de datos de entrenamiento. Dado que procesar todos los datos a la vez puede ser inviable computacionalmente, se divide en *minibatches* que permiten actualizar los pesos de forma incremental. El *batch size* afecta tanto la calidad del gradiente como el coste computacional: a) valores pequeños introducen más ruido, pero facilitan escapar de óptimos locales; b) valores grandes producen gradientes más estables, pero requieren más memoria y pueden converger en soluciones subóptimas [11].

2.3.2. Tasa de aprendizaje y *weight decay*

La tasa de aprendizaje (*learning rate*) controla el tamaño de los pasos en la actualización de los parámetros. Una tasa inadecuada puede causar inestabilidad o una convergencia lenta [5]. El *weight decay*, técnica de regularización que penaliza pesos excesivos, ayuda a reducir el sobreajuste mediante una forma de regularización L2 [12].

2.3.3. Función de pérdida

La función de pérdida mide el error entre las predicciones y las etiquetas reales. En este proyecto, centrado en una tarea de clasificación binaria, se emplea la entropía cruzada binaria:

$$\mathcal{L} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

2.3.4. *Overfitting* y técnicas de regularización

El fenómeno de *overfitting* se da cuando un modelo ajusta excesivamente sus parámetros a los datos de entrenamiento, llegando a capturar también el ruido y los patrones no representativos del problema subyacente. Esta situación compromete su capacidad de generalización a nuevos datos. Para prevenir o mitigar el *overfitting*, se emplean diversas técnicas, entre las cuales se encuentran:

- *Dropout*: desactiva aleatoriamente neuronas durante el entrenamiento [13].
- Regularización L1/L2: penaliza pesos grandes.
- *Early stopping*: detiene el entrenamiento si el rendimiento en validación deja de mejorar.
- *Data augmentation*: en PLN, puede incluir técnicas como la sustitución de sinónimos o traducción inversa.

2.3.5. *Early stopping* y validación

El *early stopping* consiste en finalizar el entrenamiento cuando la métrica de validación se estanca o empeora tras un número determinado de épocas [14]. Esta técnica es complementaria a una correcta partición de los datos en conjuntos de entrenamiento, validación y prueba, garantizando una evaluación fiable del rendimiento y evitando el sobreajuste.

2.4. Procesamiento del lenguaje natural (PLN)

El PLN es otra subdisciplina de la inteligencia artificial orientada al diseño de modelos computacionales capaces de comprender, procesar y generar lenguaje humano en contextos reales. En este trabajo, el PLN desempeña un papel esencial, ya que la tarea principal se concentra en analizar textos descriptivos de proyectos de *crowdfunding*.

2.4.1. Preprocesamiento del texto

El preprocesamiento es un paso fundamental para preparar los textos antes de que sean introducidos en modelos de AA. Este proceso incluye una serie de transformaciones, cuyo objetivo es reducir el ruido y normalizar el lenguaje:

- **Limpieza del texto:** eliminación de caracteres especiales, puntuación innecesaria o etiquetas HTML.
- **Tokenización:** segmentación del texto en unidades significativas llamadas *tokens*.
- **Normalización:** conversión del texto a minúsculas, eliminación de tildes, etc.
- **Eliminación de *stopwords*:** supresión de palabras frecuentes sin carga semántica significativa.
- **Lematización:** reducción de las palabras a su forma base para un análisis más coherente.

Estas operaciones buscan homogeneizar la entrada textual para facilitar la tarea de aprendizaje por parte de los modelos [15].

2.4.2. Representación del texto

Los algoritmos de AA requieren que los datos de entrada estén en formato numérico. Para ello, el texto debe ser convertido en vectores que representen su contenido semántico. Algunas técnicas clásicas son, principalmente:

- ***Bag of Words* (BoW):** Representa la frecuencia de aparición de palabras sin tener en cuenta el orden.
- **TF-IDF:** Ajusta la frecuencia de las palabras en función de su relevancia dentro del corpus.

Estas técnicas, sin embargo, no capturan el contexto ni el significado completo de las palabras. Para superarlo, se desarrollaron *word embeddings* como Word2Vec

[16], GloVe [17] o FastText [18], que permiten representar palabras como vectores densos que reflejan sus relaciones semánticas.

Más recientemente, los modelos Transformers [10] y sus variantes han introducido representaciones contextuales, donde el significado de una palabra varía según su contexto. Este enfoque ha mejorado sustancialmente el rendimiento en tareas de clasificación de texto [19, 20], como la planteada en este trabajo.

2.4.3. Clasificación de texto en PLN

La clasificación de texto es una tarea fundamental en PLN, que consiste en asignar una o varias etiquetas a un documento. Existen diferentes enfoques según el tipo de etiquetado:

- **Clasificación binaria:** el texto pertenece o no a una categoría. Es el caso de este trabajo, por ejemplo, se predice si un proyecto es exitoso o no.
- **Clasificación multiclase:** se asigna una única categoría entre varias posibles.
- **Clasificación multietiqueta:** se permiten múltiples etiquetas simultáneamente para una misma instancia.

Sobre esta base, en este proyecto se lleva a cabo una clasificación binaria supervisada, utilizando descripciones textuales de proyectos de *crowdfunding* previamente etiquetados como exitosos o no exitosos, con el objeto de predecir dicha etiqueta mediante modelos de lenguaje avanzados como los Transformer.

2.5. Transformers: fundamentos, estructura y aplicaciones

2.5.1. Introducción

La arquitectura Transformer ha supuesto un punto de inflexión en el campo del *deep learning*, especialmente en tareas de PLN. Introducida por Vaswani et al. [10], esta arquitectura reemplazó a las RNN y sus variantes, gracias a su capacidad para modelar relaciones de largo alcance mediante el mecanismo de *self-attention*, que eliminaba la necesidad de recurrencia. Su eficiencia y escalabilidad han impulsado el desarrollo de modelos de lenguaje preentrenados de gran impacto, como BERT, GPT y T5 [19, 21, 22], así como variantes adaptadas al español como BETO¹ y RoBERTa-base-bne², resultando procesamientos optimizados para acometer el objeto de este trabajo.

¹<https://huggingface.co/dccuchile/bert-base-spanish-wwm-uncased>

²<https://huggingface.co/PlanTL-GOB-ES/roberta-base-bne>

2.5.2. Contexto: limitaciones de las arquitecturas previas

Antes de la irrupción de los Transformers, las RNN, LSTM y GRU eran los modelos predominantes para tareas secuenciales. Si bien, presentaban limitaciones como, por ejemplo, la dificultad para capturar dependencias de largo alcance debido al problema del desvanecimiento del gradiente [23], así como una capacidad limitada para ser paralelizados [24]. El mecanismo de atención introducido por los Transformers superó estas barreras al evaluar relaciones entre todos los elementos de una secuencia de forma simultánea y eficiente [10].

2.5.3. Mecanismo de *self-attention*

El componente central del Transformer es el mecanismo de *self-attention*, que permite a cada palabra influir en el procesamiento de las demás, independientemente de su posición. Este mecanismo transforma cada *token* de entrada en una triple proyección: consultas (Q), claves (K) y valores (V), y calcula la atención como:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

donde d_k es la dimensión de las claves. La atención multi-cabeza (*multi-head attention*) facilita al modelo aprender diferentes representaciones simultáneamente [25].

2.5.4. *Word embeddings*: representación semántica del lenguaje

Los *word embeddings* representan las palabras como vectores densos en espacios semánticos. Aunque técnicas como Word2Vec [16] y GloVe [17] ofrecían representaciones fijas, los Transformers introdujeron *embeddings contextuales* que adaptan el vector de cada palabra a su contexto, como ocurre en ELMo [26] y BERT [19], lo cual resulta crucial en tareas referidas a la clasificación contextual de texto, véase, por ejemplo, en proyectos publicados en plataformas de *crowdfunding*.

2.5.5. *Positional encoding*

Dado que los Transformers procesan las secuencias en paralelo, necesitan incorporar explícitamente el orden de las palabras. Para ello, se suman a los *embeddings* unas codificaciones posicionales que pueden ser fijas (sinusoidales, como en el trabajo original [10]) o aprendidas (como en BERT [19]).

2.5.6. Arquitectura general del transformer

A este respecto, la arquitectura Transformer (Figura 2.1 extraída del estudio [10]) se divide en dos bloques: *encoder* y *decoder*. Cabe resaltar que en tareas de clasificación de texto como la planteada en este trabajo se emplea principalmente el *encoder* que se detalla a continuación.

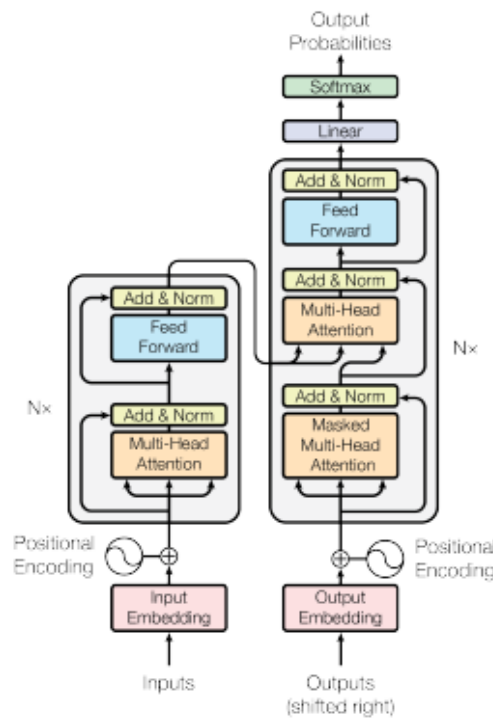


Figura 2.1: Arquitectura Transformer.

Codificador (*encoder*)

El *encoder* está formado por capas repetidas de:

- **Multi-head attention**, que permite atender a múltiples relaciones semánticas simultáneamente.
- **Redes feed-forward (FFN)**, que aplican transformaciones no lineales independientes a cada posición.
- **Conexiones residuales y normalización por capas (*LayerNorm*)**, que estabilizan y aceleran el entrenamiento [27].

Decodificador (*decoder*)

El decodificador, empleado en tareas generativas (razón por la que se descarta en este proyecto), introduce atención enmascarada (*masked attention*) y atención cruzada sobre las salidas del *encoder*.

Capa de salida

En clasificación, la salida del *encoder* se desarrolla esencialmente por una capa de proyección lineal seguida de una *softmax*, que devuelve la probabilidad de cada clase.

2.5.7. Limitaciones y retos actuales

La principal limitación de los Transformers se adscribe a su complejidad cuadrática en la longitud de la secuencia debido al mecanismo de atención, lo que dificulta su aplicación en textos largos [28]. Además, su entrenamiento requiere una gran cantidad de datos y recursos computacionales [29]. Para mitigar estas limitaciones, se han propuesto variantes como Linformer [30], Longformer [31] y Performer [32], que optimizan la atención para escalabilidad.

2.6. Modelos seleccionados: BETO y RoBERTa-base-bne

Como se señalaba, en este trabajo se han aplicado modelos de lenguaje preentrenados basados en la arquitectura Transformer. En concreto, BETO y RoBERTa-base-bne. Ambos han sido seleccionados por su alto rendimiento en tareas de PLN en español, su disponibilidad para el ajuste fino (*fine-tuning*) y su adecuación a los objetivos de clasificación del proyecto. Estos derivan del modelo BERT [19], presentando diferencias en su preentrenamiento y adaptación lingüística, lo que los hace especialmente relevantes para el entorno hispanohablante.

2.6.1. RoBERTa: *Robustly optimized BERT pretraining approach*

RoBERTa fue propuesto por Liu et al. [33] como una optimización del modelo BERT. Aunque mantiene su arquitectura base, introduce mejoras fundamentales en el proceso de preentrenamiento:

- Eliminación de la tarea de *Next Sentence Prediction* (NSP).
- Entrenamiento sobre un corpus significativamente mayor (160 GB frente a los 16 GB de BERT).
- Uso de técnicas como *dynamic masking*, tamaños de lote grandes y mayor número de pasos de entrenamiento.

Estas mejoras permiten a RoBERTa capturar representaciones contextuales más ricas y generalizables, superando a BERT en tareas evaluadas en el *benchmark* GLUE [33]. Para este trabajo se utiliza la variante RoBERTa-base-bne, entrenada específicamente con datos en español, siendo el lenguaje principal de los proyectos de Goteo.

2.6.2. RoBERTa-base-bne

RoBERTa-base-bne es una versión de RoBERTa entrenada exclusivamente en español por el Barcelona Supercomputing Center (BSC) [34], que emplea corpus como Spanish Billion Words, OPUS y textos gubernamentales y de noticias.

A diferencia de modelos multilingües como mBERT, su entrenamiento especializado le permite comprender fenómenos morfosintácticos y semánticos propios del español. Ha mostrado resultados competitivos en tareas como clasificación de texto, análisis de sentimiento y etiquetado de secuencias, superando en algunos casos a modelos como BETO en *benchmarks* como MLSum, PAWS-X-es o POS-UD [35].

2.6.3. BETO: BERT para el español

BETO es un modelo basado en la arquitectura BERT, entrenado, también, con textos en español. Fue desarrollado por Cañete et al. [20] usando corpus como Wikipedia en español y OPUS. Su configuración replica la de BERT-Base: 12 capas, 768 dimensiones ocultas y 12 cabezas de atención.

BETO ha demostrado un rendimiento notable en tareas de PLN como clasificación, reconocimiento de entidades y análisis de sentimiento. Su carácter monolingüe le posibilita competir con modelos multilingües de mayor tamaño, manteniendo un menor coste computacional, lo que lo convierte en una alternativa eficiente para tareas especializadas relacionadas con la clasificación de proyectos de *crowdfunding*.

2.7. Aprendizaje por transferencia

El aprendizaje por transferencia se ha consolidado como una técnica esencial en el desarrollo de sistemas modernos de inteligencia artificial, especialmente en tareas de PLN o de visión por computador. Su premisa fundamental es que los conocimientos adquiridos por un modelo en una tarea inicial pueden reutilizarse para mejorar el rendimiento en una tarea diferente, incluso si se dispone de pocos datos etiquetados [36, 37].

En PLN, esta estrategia ha cobrado especial importancia con el auge de modelos de lenguaje preentrenados como BERT [19], RoBERTa [33] o T5 [22], que se entrenan inicialmente en grandes corpus mediante tareas auto-supervisadas, como el *masked language modeling* (MLM). Estos modelos capturan representaciones semánticas profundas y generalistas del lenguaje, que luego pueden adaptarse a tareas específicas como clasificación de texto, análisis de sentimiento o detección de sesgos. Este enfoque resulta especialmente útil en contextos con escasez de datos o lenguas menos representadas, como la española, por lo que ha sido clave en este trabajo para

la clasificación binaria de proyectos de *crowdfunding*, permitiendo reutilizar modelos previamente entrenados y especializados como BETO [20] o RoBERTa-base-bne [34].

2.7.1. *Fine-tuning*

Una de las técnicas más eficaces dentro del aprendizaje por transferencia es el *fine-tuning*, que consiste en reentrenar parcialmente un modelo preentrenado sobre una tarea específica. A diferencia del uso estático como extractor de características, el *fine-tuning* permite que el modelo actualice sus representaciones internas y se adapte al dominio de destino mediante aprendizaje supervisado [38].

Durante este proceso, es común incorporar una nueva capa de salida específica a la tarea, congelar algunas capas inferiores del modelo para evitar sobreajuste y ajustar solo las capas superiores. Esta estrategia es muy útil cuando se dispone de un conjunto de datos reducido o ruidoso.

Ahora bien, el ajuste de hiperparámetros es crítico para garantizar el éxito del *fine-tuning*. Elementos como la tasa de aprendizaje, el tamaño de lote, el número de épocas o las técnicas de regularización influyen significativamente en la estabilidad y capacidad de generalización del modelo [39, 40]. Una elección inadecuada puede provocar que el modelo olvide el conocimiento adquirido en la fase de preentrenamiento o que no se adapte correctamente a la nueva tarea. Gracias al *fine-tuning*, los modelos utilizados en este trabajo han sido adaptados a la tarea de clasificación binaria de proyectos, logrando un rendimiento competitivo incluso bajo condiciones de desequilibrio entre clases y recursos computacionales limitados.

2.8. *Ensemble* de modelos

Las técnicas de *ensemble* constituyen una estrategia muy común en AA para mejorar el rendimiento predictivo y la robustez de los modelos. En lugar de entrenar un único modelo, el enfoque *ensemble* combina múltiples modelos base con el objetivo de reducir la varianza, el sesgo o mejorar la estabilidad del sistema global [41].

Esta combinación de modelos busca explotar la diversidad entre clasificadores para obtener predicciones más fiables. En tareas de clasificación de texto, análisis de sentimiento o detección de anomalías, los *ensembles* suelen ofrecer una mejora significativa respecto a modelos individuales, especialmente cuando el conjunto de datos es complejo, ruidoso o presenta desbalance entre clases [42].

2.8.1. *Ensemble simple*

Una de las formas más accesibles de combinación es el *ensemble simple*, también conocido como votación mayoritaria (*hard voting*). En este método, cada modelo realiza una predicción independiente y la clase final se decide por mayoría.

A pesar de su simplicidad, puede ofrecer mejoras sustanciales cuando los modelos individuales son suficientemente diversos y presentan errores no correlacionados [43, 44]. Su bajo coste computacional y su facilidad de implementación lo convierten en un procedimiento útil en contextos que cuentan con recursos limitados.

2.8.2. Tipos de *ensemble*

Bagging (bootstrap aggregating): esta técnica consiste en generar múltiples subconjuntos de datos mediante muestreo aleatorio con reemplazo (*bootstrap*) y entrenar un modelo independiente en cada subconjunto. Las predicciones se agregan mediante votación (en clasificación) o promediado (en regresión). Este enfoque reduce la varianza del modelo y mejora la estabilidad, en especial, en clasificadores de alta varianza, como los árboles de decisión.

Underbagging: variante del *bagging* orientada a problemas con clases desbalanceadas. En lugar de muestrear con reemplazo sobre todo el conjunto, se submuestra aleatoriamente la clase mayoritaria en cada iteración para equilibrarla con la clase minoritaria. Esta técnica ha demostrado ser efectiva en escenarios donde las clases relevantes son infrecuentes y los datos están significativamente desbalanceados [45, 46]. En este trabajo, se emplea *underbagging* para entrenar tres modelos independientes por cada campo textual, generando subconjuntos balanceados a partir del conjunto de entrenamiento. Posteriormente, los resultados se combinan mediante votación mayoritaria, generando una predicción única por campo.

Boosting: a diferencia del *bagging*, el *boosting* entrena modelos de manera secuencial. Cada nuevo modelo se especializa en corregir los errores del anterior, asignando mayor peso a los ejemplos mal clasificados. Técnicas como *AdaBoost*, *Gradient Boosting* o *XGBoost* son ampliamente utilizadas por su capacidad para reducir el sesgo y lograr alta precisión [47].

Stacking (stacked generalization): este enfoque combina varios modelos, cuyos resultados sirven de entrada para un modelo meta (o de nivel superior), que aprende a optimizar la combinación final. A diferencia del *bagging* y el *boosting*, el *stacking* puede capturar interacciones complejas entre modelos [48].

2.8.3. *Ensembles* con modelos de lenguaje

El enfoque *ensemble* ha sido adoptado con éxito en PLN, específicamente con modelos Transformer como BERT, RoBERTa o BETO. Dado que estos modelos se entrenan con diferentes inicializaciones, corpus o configuraciones, combinarlos puede reducir su varianza y mejorar el rendimiento global [40].

Por esta razón, en este trabajo se ha aplicado un sistema jerárquico de *ensembles*. Por cada uno de los cuatro campos textuales relevantes (*about*, *goal*, *motivation*, *description*), se entrenan tres modelos independientes utilizando *underbagging*. Las predicciones de cada trío de modelos se agregan mediante votación mayoritaria, generando una salida única por campo. Finalmente, las salidas de los cuatro campos se combinan también por votación para obtener la predicción final del sistema. Con este enfoque se permite capturar en mejores condiciones la señal semántica de cada campo textual, adaptándose, además, a la limitación de tokens de los modelos y al desbalanceo presente en el conjunto de datos.

2.9. Interpretabilidad de modelos de AA

A medida que los modelos de AA se vuelven más complejos, fundamentalmente en el ámbito del PLN, surge la necesidad de comprender las decisiones que toman dichos modelos. Esta necesidad da lugar al campo de la interpretabilidad, cuyo objetivo es proporcionar explicaciones comprensibles sobre el funcionamiento interno de los modelos o las razones subyacentes a sus predicciones.

La interpretabilidad es importante en áreas como la medicina, el derecho, las finanzas y, en el contexto de este trabajo, la evaluación de proyectos de *crowdfunding* (empresa y administración), donde comprender por qué un modelo predice el éxito o fracaso de una campaña puede tener implicaciones prácticas para los usuarios y/o las plataformas. Existen dos enfoques predominantes para abordar la interpretabilidad:

- Modelos intrínsecamente interpretables: modelos simples como árboles de decisión o regresión lineal, cuyo comportamiento puede analizarse directamente.
- Métodos *post-hoc*: técnicas aplicadas a modelos complejos (como redes neuronales profundas o Transformers) para analizar y explicar sus decisiones tras el entrenamiento.

Entre los métodos *post-hoc* más destacados se encuentran LIME [49], SHAP [50] e *Integrated Gradients* [51], cada uno de los cuales cuenta con ventajas que dependen de diferentes contextos. En el caso de los modelos Transformer, la interpretabilidad se ha beneficiado del uso de visualizaciones de atención, análisis de gradientes y herramientas de atribución de relevancia para entradas textuales.

2.9.1. *Integrated Gradients*

Integrated Gradients es una técnica de atribución introducida por Sundararajan et al. [51] que posibilita la una estimación rigurosa de la contribución de cada entrada en la predicción de un modelo. Esta técnica ha sido ampliamente empleada en modelos complejos relacionados con redes neuronales profundas y arquitecturas Transformer, debido a su solidez teórica y a la estabilidad de sus resultados frente a pequeñas perturbaciones.

El planteamiento parte de la idea de medir cuánto influye cada componente de la entrada (en el caso del PLN, cada *token*) en la salida final del modelo. Para ello, se define un punto de referencia o entrada base (una versión vacía o neutral del ejemplo de entrada) y se calcula cómo varía la predicción del modelo a medida que se transita desde esa entrada base hasta la entrada real. Este recorrido se modela como un camino lineal en el espacio de características, sobre el cual se computa una integral de los gradientes. Formalmente, la atribución para una característica x_i se define como:

$$\text{IG}_i(x) = (x_i - x'_i) \cdot \int_{\alpha=0}^1 \frac{\partial F(x' + \alpha(x - x'))}{\partial x_i} d\alpha$$

donde:

- x es la entrada real,
- x' es la entrada base,
- F es la función que representa el modelo,
- $\alpha \in [0, 1]$ parametriza el camino entre x' y x .

Esta integral se aproxima numéricamente mediante una suma discreta. El resultado es una puntuación para cada token, que cuantifica su influencia sobre la decisión del modelo.

Una de las principales ventajas de *Integrated Gradients* es que cumple propiedades deseables como la *sensibilidad* (*tokens* con impacto observable deben recibir puntuaciones distintas de cero) y la *aditividad* (la suma de las atribuciones es igual al cambio en la predicción entre la entrada base y la entrada real). Estas garantías matemáticas la convierten en una técnica fiable para tareas de análisis interpretativo.

En este trabajo, *Integrated Gradients* se aplica para analizar modelos Transformer entrenados sobre campañas de *crowdfunding*, lo que facilita identificar qué partes del texto son más relevantes en la predicción del éxito o el fracaso.

2.9.2. Interpretabilidad con Captum

Captum³ es una biblioteca desarrollada por Facebook AI que proporciona una plataforma unificada para la interpretabilidad de modelos en PyTorch [52]. Captum implementa diversas técnicas de atribución, incluyendo:

- *Integrated Gradients*
- *Gradient SHAP*
- *DeepLIFT*
- *Layer Conductance*

Estas técnicas permiten cuantificar la contribución de cada entrada (por ejemplo, cada *token* de texto) a la predicción final del modelo. En modelos Transformer como BERT o RoBERTa, Captum posibilita la comprensión del comportamiento del modelo al visualizar qué partes del texto han influido más en la decisión del clasificador.

En este proyecto se utiliza Captum con la finalidad de analizar las predicciones de modelos Transformer entrenados sobre las descripciones de campañas de *crowdfunding*. Esto es, para identificar los fragmentos de texto más relevantes en las decisiones del modelo, lo cual contribuye tanto a validar el comportamiento del sistema como a dotar de transparencia a su funcionamiento.

Captum ha sido validado en la literatura como una herramienta eficaz para tareas de clasificación de texto. Por ejemplo, [53] emplean Captum para evaluar la robustez de BERT en tareas de análisis de sentimiento, mostrando que las visualizaciones de atribuciones pueden revelar sesgos o errores aprendidos por el modelo.

2.10. Medidas de evaluación

La evaluación del rendimiento de los modelos de aprendizaje automático constituye una parte esencial del desarrollo de sistemas de PLN. Las métricas permiten cuantificar el grado de acierto de un modelo y compararlo con otras alternativas, dando lugar a decisiones objetivas durante la validación, selección e implementación del sistema [54].

En este trabajo, centrado en tareas de clasificación de texto, se emplean métricas derivadas de la matriz de confusión, tales como la precisión, la sensibilidad, la medida F1 y la exactitud. Estas métricas no solo informan del número total de aciertos, sino que delimitan cómo se distribuyen los errores, lo cual es especialmente relevante en contextos con desbalance entre clases.

³<https://captum.ai/>

2.10.1. Matriz de confusión

La matriz de confusión es una representación tabular que resume las predicciones del modelo frente a los valores reales. En problemas de clasificación binaria, incluye:

- **Verdaderos Positivos (VP)**: casos positivos correctamente clasificados.
- **Falsos Positivos (FP)**: casos negativos clasificados erróneamente como positivos.
- **Verdaderos Negativos (VN)**: casos negativos correctamente clasificados.
- **Falsos Negativos (FN)**: casos positivos clasificados erróneamente como negativos.

2.10.2. Precisión

La precisión (*precision*) indica la proporción de predicciones positivas que son correctas:

$$\text{Precisión} = \frac{VP}{VP + FP}$$

Es principalmente útil cuando se desea minimizar los falsos positivos [55].

2.10.3. Sensibilidad o recobrado

La sensibilidad (*recall*) mide la capacidad del modelo para identificar los positivos reales:

$$\text{Sensibilidad} = \frac{VP}{VP + FN}$$

Es fundamental en escenarios donde omitir casos relevantes tiene un alto coste, como en detección de fraude o diagnóstico médico [56].

2.10.4. Medida F1

La medida F1 es la media armónica entre precisión y sensibilidad:

$$F1 = 2 \cdot \frac{\text{Precisión} \cdot \text{Sensibilidad}}{\text{Precisión} + \text{Sensibilidad}}$$

Resulta adecuada en contextos con clases desbalanceadas.

2.10.5. Exactitud

La exactitud (*accuracy*) representa el porcentaje total de predicciones correctas:

$$\text{Exactitud} = \frac{VP + VN}{VP + VN + FP + FN}$$

Aunque es una métrica intuitiva, puede resultar engañosa cuando existe un fuerte desequilibrio entre clases [57].

2.10.6. Curva ROC y AUC

La curva ROC (*Receiver Operating Characteristic*) representa la relación entre la sensibilidad y la tasa de falsos positivos para diferentes umbrales de decisión [56, 58]. El área bajo esta curva (AUC-ROC) es una medida global del rendimiento del modelo:

$$\text{AUC} \in [0, 1], \quad \text{siendo } 1 \text{ el valor ideal}$$

Un AUC más cercano a 1 indica mayor capacidad del modelo para distinguir entre clases.

2.11. Trabajos relacionados

El estudio de campañas de *crowdfunding* ha ido adquiriendo relevancia en la literatura científica, en especial en lo que respecta a la integración de características textuales y modelos de AA para predecir la probabilidad de éxito de los proyectos. En este marco, diferentes investigaciones han planteado el problema desde perspectivas y escenarios divergentes, incorporando datos multimodales, técnicas de representación lingüística avanzada y métodos de aprendizaje profundo variados.

Chan et al. [59] compararon métodos basados en PLN con modelos tradicionales de análisis textual para predecir el éxito de campañas de *crowdfunding* en la plataforma global *Kickstarter*, con sede en Nueva York, y sostenida sobre proyectos relacionados con la creatividad (música, arte, fotografía y otros). En el estudio se aplicaron modelos BERT para calcular *scores* de similaridad semántica y los confrontaron con índices clásicos de legibilidad.

Así, los resultados revelaron que las representaciones obtenidas mediante BERT ofrecen una mayor capacidad discriminativa frente a métricas sintácticas, subrayando la eficacia de los modelos de lenguaje profundo en la extracción de patrones

implícitos en textos promocionales. Además, los autores observaron que la complejidad textual medida por métodos clásicos no siempre se correlaciona con el éxito, lo que refuerza la necesidad de abordar enfoques más contextualizados mediante aprendizaje profundo.

Adicionalmente, Zhang y Lau [60] evaluaron la predicción del éxito en campañas de *crowdfunding* desde una perspectiva multimodal, integrando texto, metadatos y señales visuales. A través de una arquitectura híbrida basada en redes neuronales profundas, los autores fusionaron descripciones textuales codificadas con BERT, imágenes de campaña procesadas con CNNs y variables estructurales.

Su experimentación demostró que el uso de características multimodales mejora de forma significativa la capacidad predictiva respecto a modelos univariados, destacando el valor añadido de incorporar elementos visuales que capturan dimensiones persuasivas no lingüísticas propias de la proyección de una campaña. El trabajo pone de manifiesto la eficacia de la fusión de modalidades en entornos donde la percepción visual y narrativa juegan roles complementarios en la toma de decisiones del usuario.

En una línea de investigación más centrada en la comparación de representaciones textuales, Gunduz [61] realizó un análisis comparativo entre modelos de lenguaje basados en FastText y BERT para la clasificación binaria del éxito de este tipo de campañas. A través de un conjunto de experimentos basados en proyectos en inglés, concluía que las representaciones contextuales aprendidas mediante BERT superan sistemáticamente a los *embeddings* estáticos de FastText en métricas de precisión, F1 y AUC.

Asimismo, el autor enfatizó la importancia del *fine-tuning* sobre el dominio específico como mecanismo de especialización, ya que incrementa la adaptabilidad del modelo a las características discursivas particulares del entorno de *crowdfunding*.

En suma, estos estudios convergen en tres aportaciones fundamentales: la relevancia de los modelos de lenguaje profundo como herramientas efectivas para extraer información semántica de textos persuasivos; el impacto positivo de la multimodalidad en la predicción del éxito, al integrar información textual, visual y estructural; y la necesidad de incorporar técnicas de especialización e interpretabilidad para adaptar estos modelos a contextos específicos y hacer más transparente su comportamiento.

2.12. Tecnologías y recursos utilizados

El desarrollo de este trabajo se ha sustentado en un ecosistema tecnológico orientado al PLN mediante técnicas de *deep learning*. La elección de cada herramienta ha respondido a criterios de eficiencia, flexibilidad, comunidad de soporte y compatibilidad

con los modelos actuales basados en arquitecturas Transformer (Figura 2.2).

Python ha constituido el lenguaje base para toda la implementación, dada su versatilidad y el amplio ecosistema de bibliotecas para ciencia de datos, AA y PLN. Sobre esto, se ha desarrollado la lógica de preprocesamiento, entrenamiento, evaluación, interpretabilidad y gestión de experimentos. También, y a fin de facilitar el desarrollo exploratorio y la validación interactiva de los modelos, se ha recurrido a **Jupyter Notebook**, una herramienta ampliamente adoptada en investigación y docencia, que ha permitido documentar el flujo de trabajo de forma estructurada y reproducible.

El núcleo del sistema de entrenamiento ha sido implementado mediante **PyTorch**, un *framework* de aprendizaje profundo que ofrece flexibilidad en la construcción de arquitecturas y control detallado sobre el entrenamiento. PyTorch ha facilitado tanto la integración de modelos preentrenados como la personalización de etapas de *fine-tuning* y evaluación.

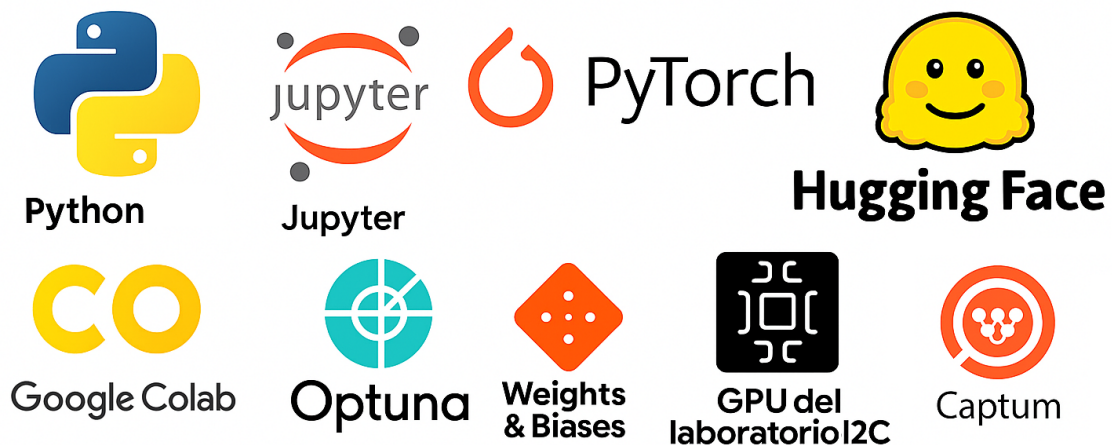


Figura 2.2: Tecnologías y recursos utilizados en el desarrollo del trabajo.

Otra pieza clave del proyecto ha sido el uso de la biblioteca **Hugging Face Transformers**⁴, que ha facilitado el acceso a los modelos de lenguaje preentrenados BERT y RoBERTa-base, integrándolos al *pipeline* de entrenamiento y explotando su compatibilidad con técnicas de aprendizaje por transferencia. Esta biblioteca ha favorecido, también, la *tokenización* eficiente de textos y la gestión de arquitecturas complejas sin necesidad de reimplementación desde cero.

Parte del desarrollo del código y las pruebas del mismo se han desarrollado en la plataforma en la nube **Google Colab**⁵, que ha ofrecido acceso gratuito a GPUs,

⁴<https://huggingface.co>

⁵<https://colab.research.google.com>

permitiendo pruebas sin recursos locales. No obstante, parte del entrenamiento final y más intensivo se ha ejecutado en entornos propios del grupo de investigación, concretamente en una **GPU del laboratorio I2C**, con el objetivo de disponer de mayor control sobre los tiempos de ejecución y evitar las restricciones impuestas por plataformas externas.

La optimización de los modelos se ha beneficiado del uso combinado de **Optuna**⁶ y **Weights & Biases**⁷ (**W&B**). Optuna se ha empleado como motor de búsqueda eficiente de hiperparámetros mediante algoritmos de optimización bayesiana, mientras que W&B ha servido como herramienta de seguimiento, visualización y trazabilidad de experimentos, permitiendo comparar diversas configuraciones, así como analizar el proceso de entrenamiento.

Finalmente, para el análisis de interpretabilidad de los modelos se ha utilizado la biblioteca **Captum**, una extensión nativa de PyTorch diseñada para estimar la importancia de cada *token* en las decisiones del modelo. Esta herramienta ha sido fundamental para llevar a cabo análisis de atribución sobre modelos como BETO y RoBERTa, empleando técnicas como *Integrated Gradients* y generando visualizaciones comprensibles de los fragmentos textuales que han influido más significativamente en las predicciones.

En este orden, la integración de estas tecnologías ha dado lugar a la construcción de un sistema completo, reproducible y científicamente escalable, alineado con los estándares actuales en investigación en PLN y AA. Su aplicación ha sido decisiva tanto para implementar los modelos como para comprender su funcionamiento interno y evaluar su rendimiento.

⁶<https://optuna.org>

⁷<https://wandb.ai/site/>

Metodología, experimentación y resultados

En este capítulo se detalla el enfoque metodológico para abordar la tarea de predicción del éxito de proyectos de *crowdfunding* a partir de información textual de Goteo. La estrategia que sigue se fundamenta en el uso de modelos de lenguaje pre-entrenados aplicados sobre diferentes campos semánticos disponibles en un proyecto, junto con técnicas específicas para afrontar el desbalanceo de clases y las limitaciones de longitud en los modelos Transformer.

El capítulo se estructura en secciones que reflejan las fases del proceso: desde la preparación del conjunto de datos y el preprocesamiento textual, hasta el entrenamiento de los modelos, la búsqueda de hiperparámetros, la combinación mediante *ensemble* y la interpretación cuantitativa y cualitativa de los resultados.

Asimismo, se describen las herramientas, las decisiones técnicas y las medidas adoptadas para evaluar el rendimiento de los modelos. Finalmente, se discuten los resultados obtenidos y se presentan observaciones interpretativas sobre los patrones aprendidos por los modelos.

3.1. Descripción general de la metodología

El marco de trabajo de este proyecto se ha estructurado en una serie de fases orientadas al desarrollo de un sistema de clasificación binario capaz de predecir el éxito de campañas de *crowdfunding* a partir de descripciones textuales promovidas en la plataforma en español Goteo. Este proceso ha incluido tareas de preprocesamiento, generación de subconjuntos equilibrados, entrenamiento y validación de modelos basados en Transformers, combinación de predicciones mediante técnicas de *ensemble* y análisis de interpretabilidad.

La metodología partió del conjunto de datos original, que se dividió en dos subconjuntos: datos de entrenamiento y datos de prueba. Sobre los datos de entrenamiento se realizó una búsqueda de la proporción óptima para balancear las clases, dado el fuerte desbalance existente. Posteriormente, se aplicaron técnicas de *un-*

derbagging, generando múltiples subconjuntos (*bootstraps*), equilibrados mediante submuestreo de la clase mayoritaria.

Cada subconjunto sirvió como base para entrenar modelos independientes. Durante esta etapa, se realizó una búsqueda de hiperparámetros utilizando la herramienta de optimización Optuna y se evaluó el rendimiento en validación. Los mejores modelos obtenidos se combinaron, a su vez, mediante votación mayoritaria para generar una predicción única por campo.

Una vez finalizado el entrenamiento, los modelos se evaluaron sobre el conjunto de test, produciendo predicciones que son fueron interpretadas mediante la técnica de atribución *Integrated Gradients*. Esta etapa facilitó la identificación de los tokens más relevantes para la decisión del modelo, aumentando la transparencia del sistema. La Figura 3.1 muestra el *pipeline* del proceso descrito.

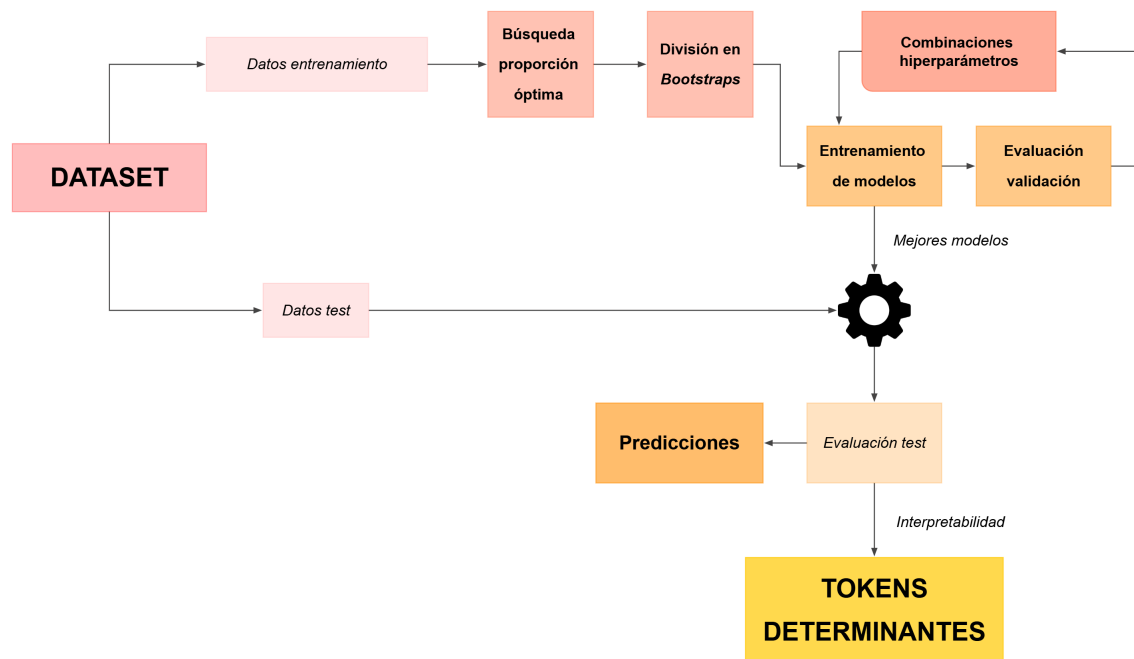


Figura 3.1: *Pipeline* del marco de experimentación.

3.2. Extracción de los datos

La obtención del conjunto de datos se realizó accediendo a la API¹ pública de la plataforma Goteo. En una primera etapa, se recopiló un listado completo de los proyectos publicados entre el 1 de enero de 2012 y el 31 de diciembre de 2023. Para este fin, se efectuaron consultas paginadas que favorecieron la recuperación

¹<https://api.goteo.org/v1/projects/>

de la totalidad de los registros disponibles en ese intervalo, respetando los límites establecidos por el servidor mediante pausas entre peticiones.

Tras reunir la base de proyectos, se procedió a una segunda fase, en la que se solicitó información detallada de cada uno de proyectos por medio de su identificador único. Este proceso permitió enriquecer el conjunto de datos con variables adicionales, tales como descripciones, fechas, metas de financiación y estado del proyecto. Finalmente, la información fue almacenada en formato JSON para facilitar su posterior análisis.

3.3. Descripción del conjunto de datos

Para este proyecto se tomó como base el conjunto de datos recopilado de la plataforma de financiación colectiva Goteo.org, centrada en proyectos con componente social, cultural y/o de innovación abierta. El *dataset* original estuvo compuesto por 2.352 proyectos de *crowdfunding* finalizados, con información detallada sobre aspectos relevantes de cada campaña.

Cada proyecto incluía variables estructuradas y no estructuradas. Las primeras correspondían a datos numéricos, categóricos o temporales (como fechas de inicio y cierre, cantidades recaudadas o categorías temáticas). Las segundas integraban campos textuales descriptivos, que representaron el núcleo comunicativo del proyecto, constituyéndose como fuente principal de información para la predicción, sostenida sobre los estudios relacionados expuestos previamente.

En la *Tabla 3.1* se muestra un resumen de los campos más relevantes del conjunto de datos:

Campo	Descripción
<code>id</code>	Identificador único del proyecto en la plataforma
<code>name</code>	Título del proyecto
<code>about</code>	Descripción general y motivación del proyecto
<code>description</code>	Explicación detallada de objetivos y actividades
<code>goal</code>	Justificación del objetivo económico
<code>motivation</code>	Razones personales o sociales para impulsar el proyecto
<code>amount</code>	Cantidad total recaudada
<code>minimum / optimum</code>	Umbral de financiación mínima y óptima
<code>date-published / date-closed</code>	Fechas de inicio y fin de la campaña
<code>categories.name</code>	Categoría temática asignada al proyecto
<code>location.city / location.country</code>	Localización geográfica
<code>status</code>	Variable objetivo: éxito (SI) o fracaso (NO) del proyecto

Tabla 3.1: Campos principales del *dataset*.

La variable *status* se utilizó como etiqueta de salida para los modelos de clasificación, indicando si el proyecto alcanzaba la financiación mínima requerida y, por tanto, exitosa. El análisis de la distribución de esta variable reveló un marcado desbalance entre clases:

- Proyectos exitosos (SI): 1.983 instancias (84 %).
- Proyectos no exitosos (NO): 370 instancias (16 %).

Esta distribución asimétrica planteó un desafío para los modelos de AA, ya que una clase tan predominante puede inducir un sesgo en el entrenamiento, generando modelos que tienden a favorecer predicciones positivas de manera excesiva. Por este motivo, se consideró aplicar estrategias de balanceo de clases, que se describen con mayor detalle en la Sección 3.5.

Además del problema del desbalance, se identificaron otras características que afectaban al preprocesamiento, como la presencia de múltiples idiomas y la gran longitud de algunos campos textuales. Estos aspectos motivaron decisiones metodológicas específicas, como la detección y traducción de idiomas (Sección 3.4.4) y la separación de modelos por campo para evitar el truncamiento de tokens (Sección 3.4.5).

3.4. Preparación de los datos

En esta sección se describe el proceso de preparación del conjunto de datos antes de su uso en los modelos de clasificación. Estas tareas incluyeron la selección de los campos textuales más relevantes, la limpieza y normalización del texto, la detección y traducción de idiomas y un análisis de la longitud de los textos tokenizados que fundamentó la decisión de entrenar modelos separados por campo.

3.4.1. Selección de campos relevantes

Para entrenar los modelos de clasificación, se seleccionaron únicamente aquellos campos textuales directamente relacionados con el contenido y presentación de los proyectos. Los campos escogidos fueron: *about*, *description*, *goal* y *motivation*. La decisión de centrarse en estos atributos responde a la naturaleza del problema y los estudios relacionados con el proyecto: predecir el éxito de un proyecto a partir de cómo se comunica/narra. Cada uno de estos campos aportó una perspectiva divergente al proyecto:

- *about*: resumen general que condensa el objetivo principal.
- *description*: explicación sobre enfoque, actividades y propósito de la campaña.

- *goal*: justificación de la financiación solicitada.
- *motivation*: razones personales o sociales del equipo impulsor.

3.4.2. Limpieza y normalización

Con el fin de preparar los textos para su tratamiento automático, se realizó un proceso de limpieza para eliminar elementos que pudieran generar ruido o ambigüedad:

- Eliminación de URLs y enlaces incrustados.
- Supresión de caracteres especiales, como, por ejemplo, asteriscos o sintaxis de Markdown.
- Eliminación de paréntesis vacíos y otros residuos de formato.
- Eliminación de un proyecto (Proy2016) que presentaba textos multilingües incoherentes.

3.4.3. Detección de idioma

En este orden, el conjunto de datos contenía textos en varios idiomas. Aunque el *dataset* incluía un campo original denominado *lang*, que indicaba el idioma de los proyectos, se decidió validar esta información aplicando una detección automática mediante la herramienta fastText², empleando el modelo preentrenado *lid.176.ftz*.

Código ISO	Proyectos
es	1847
ca	357
en	49
gl	47
eu	39
fr	10
sv	2
pt	1
ast	1

Tabla 3.2: Distribución original de idiomas según el campo *lang*.

²<https://fasttext.cc>

Código ISO	Proyectos
es	1850
ca	369
en	46
gl	34
eu	30
pt	11
fr	10
sv	2
el	1

Tabla 3.3: Distribución de idiomas detectada con fastText.

Se observaron ligeras discrepancias entre las Tablas 3.2 y 3.3. Por ejemplo, la detección automática reducía el número de textos en gallego y euskera y detectaba un proyecto en griego (el) no etiquetado en el campo original. Estas diferencias reforzaron la necesidad de emplear detección automática como paso previo a la traducción, especialmente en un conjunto compuesto por textos cortos o escritos en más de un idioma.

3.4.4. Traducción automática al español

Con el fin de garantizar la homogeneidad lingüística del corpus y aprovechar modelos entrenados en español, se tradujeron todos los textos detectados en otros idiomas al español. Se emplearon, en particular, los siguientes métodos:

- **Modelos MarianMT (HelsinkiNLP³)** a través de la librería transformers de Hugging Face para: ca, en, gl, fr.
- **Google Translate** (vía googletrans) para: eu, pt, sv.

Esta estrategia permitió mantener todos los proyectos en el análisis, evitando sesgos por idioma o pérdida de datos valiosos.

3.4.5. Tokenización y análisis de longitud

Los textos fueron *tokenizados* utilizando los *tokenizadores* oficiales de los modelos BETO y RoBERTa en español. La *tokenización* dividió el texto en unidades mínimas (*tokens*) que luego se transformaron en IDs numéricos para ser procesados por los modelos. Como estos modelos tenían una longitud máxima de entrada de 512 *tokens*, se evaluó la longitud promedio por campo textual para determinar la viabilidad de concatenarlos. Los resultados se muestran, a continuación, en la Tabla 3.4.

³<https://huggingface.co/Helsinki-NLP>

Campo	<i>Tokens</i> promedio
about	327
goal	60
motivation	222
description	335
Total concatenado	944

Tabla 3.4: Promedio de tokens por campo textual tras la tokenización.

Como puede observarse, la concatenación de todos los campos superó el límite de entrada de los modelos, lo que provocaría truncamiento y pérdida de información. Por este motivo, se decidió entrenar un modelo independiente por cada campo, de forma que cada texto se utilizara en su totalidad y sin recortes. Adicionalmente, esta decisión se vio sustentada por la hipótesis de que los campos podían tener diferentes enfoques semánticos.

De este modo, no solo se preservó la información completa, sino que se facilitó la construcción de un sistema de *ensemble* posterior, en el que se integraron las predicciones de los modelos individuales mediante votación mayoritaria (véase Sección 2.8).

3.5. Balanceo del *dataset*

Como se expuso en la Sección 3.3, el conjunto de datos presentaba una distribución muy desigual entre clases: aproximadamente un 84% de los proyectos estaban etiquetados como exitosos (SI), mientras que solo un 16% correspondía a proyectos no exitosos (NO). Este desbalance podía comprometer el rendimiento de los modelos de clasificación, especialmente en la detección de la clase minoritaria, lo que perjudicaría las métricas del *recall* o el *F1-score* de esa clase.

3.5.1. Consideraciones sobre técnicas de balanceo

Debido a la naturaleza textual del problema, se descartaron técnicas de *oversampling*, ya que duplicar ejemplos podía inducir sobreajuste, y las técnicas de generación de texto (como *data augmentation*) podían introducir ruido semántico o inconsistencias difíciles de validar. En su lugar, se optó por aplicar *undersampling* a la clase mayoritaria durante el entrenamiento, reduciendo el número de ejemplos positivos para alcanzar una proporción controlada entre clases sin generar datos sintéticos.

3.5.2. Evaluación de proporciones de clases

Con el objetivo de identificar la proporción más adecuada para el entrenamiento, se llevó a cabo una experimentación sistemática con diferentes relaciones entre clase minoritaria y mayoritaria: 1:5, 1:4, 1:3, 1:2 y 1:1. En todos los casos se utilizó el modelo *dccuchile/bert-base-spanish-wwm-uncased* (BETO *uncased*), manteniendo constantes los hiperparámetros, de forma que el único factor variable fuera la proporción de clases.

Cada experimento consistió en entrenar un modelo por cada uno de los cuatro campos textuales seleccionados, combinando sus predicciones mediante un *ensemble* por votación mayoritaria. Así, los resultados de evaluación no se basaron en un único campo, sino en la integración de las predicciones de los cuatro modelos entrenados sobre: *about*, *goal*, *motivation* y *description*.

Hiperparámetros utilizados

La configuración empleada durante esta fase experimental se resume en la Tabla 3.5.

Hiperparámetro	Valor
Modelo base	<code>dccuchile/bert-base-spanish-wwm-uncased</code>
Tokenizador	BERT Tokenizer (<i>uncased</i>)
Learning rate	1×10^{-5}
Batch size	16
Optimizador	AdamW
Weight decay	0.01
Epochs	5

Tabla 3.5: Hiperparámetros utilizados durante la experimentación con proporciones.

3.5.3. Resultados comparativos

La Tabla 3.6 muestra los resultados obtenidos tras aplicar el *ensemble* de los cuatro modelos entrenados con cada proporción. Se utilizan varias métricas relevantes, entre ellas la AUC (área bajo la curva ROC), seleccionada como métrica principal por su robustez ante desequilibrios de clase.

Se encontró que el valor más alto de AUC se alcanzaba con la proporción 1:2, ofreciendo, además, un buen equilibrio entre sensibilidad y precisión entre clases. Aunque la proporción 1:1 incrementaba ligeramente el *recall* de la clase minoritaria, lo hacía a costa de una pérdida general de precisión. Por tanto, se seleccionó la proporción 1:2 como configuración óptima para el resto del trabajo.

Proporción	Accuracy	Prec. macro	Recall macro	F1 clase 1	F1 clase 0	AUC
5:1	0.845	0.693	0.551	0.914	0.198	0.597
4:1	0.830	0.637	0.569	0.904	0.259	0.619
3:1	0.820	0.643	0.618	0.895	0.361	0.656
2:1	0.747	0.595	0.630	0.842	0.364	0.670
1:1	0.699	0.593	0.656	0.801	0.383	0.654

Tabla 3.6: Resultados del *ensemble* por votación mayoritaria según proporción de clases.

3.5.4. División del conjunto de datos

Después de tomar la proporción 1:2 como referencia (Tabla 3.7), se estableció una división estándar del conjunto de datos:

- 80 % de los datos se utilizaron para entrenamiento (aplicando balanceo).
- 20 % se reservaron para test (sin alterar su proporción original).
- Dentro del conjunto de entrenamiento, se reservó un 20 % para validación.

Conjunto	Total	Clase 0 (mayoritaria)	Clase 1 (minoritaria)
Entrenamiento (balanceado 1:2)	1,200	800	400
Validación (balanceado 1:2)	180	120	60
Test (sin balancear)	375	300	75

Tabla 3.7: Distribución de ejemplos por conjunto tras división y balanceo (valores estimados).

El *undersampling* se aplicó sobre el conjunto de entrenamiento y validación. El test conservó la distribución real de clases, permitiendo así evaluar los modelos en condiciones representativas del dominio original. Esta estrategia facilitó el entrenamiento de los modelos con una proporción equilibrada que mejoraba la capacidad de detectar la clase minoritaria, manteniendo al mismo tiempo una evaluación realista sobre los datos no balanceados.

3.6. Modelado y entrenamiento

Esta sección describe el proceso de entrenamiento y evaluación de los modelos empleados para predecir el éxito de campañas de *crowdfunding* en Goteo. A partir de los campos textuales seleccionados (*about*, *goal*, *motivation* y *description*), se entrenaron modelos independientes por campo, debido a la imposibilidad de concatenarlos sin sobrepasar el límite de *tokens* que imponen las arquitecturas BERT y RoBERTa (512 *tokens*). La combinación de las salidas se realizó mediante técnicas de *ensemble*, tal y como se detalla a continuación.

3.6.1. Búsqueda de hiperparámetros con Optuna

Con el objetivo de obtener modelos optimizados para la tarea de predicción del éxito en campañas, se llevó a cabo una búsqueda de hiperparámetros automatizada utilizando la herramienta Optuna. Esta fase fue esencial para asegurar que cada combinación de modelo y campo textual se entrenase bajo las condiciones más favorables.

Se realizaron ocho búsquedas independientes, una por cada combinación del modelo BETO y RoBERTa-base-bne) con los cuatro campos textuales considerados: *about*, *description*, *goal* y *motivation*. El espacio de búsqueda definido para Optuna se resume en la Tabla 3.8.

Hiperparámetro	Valores posibles
<code>learning_rate</code>	{5e-6, 1e-5, 2e-5, 3e-5, 5e-5, 1e-4}
<code>weight_decay</code>	{0.1, 0.01, 0.001}
<code>optimizer</code>	{adamw_torch, adafactor, schedule_free_radam}
<code>batch_size</code>	{16, 32}

Tabla 3.8: Espacio de búsqueda de hiperparámetros con Optuna.

Cada búsqueda se llevó a cabo sobre el conjunto de entrenamiento previamente balanceado (proporción 1:2 a favor de la clase negativa), manteniendo una validación interna estratificada del 20 %. La métrica utilizada como criterio de selección fue el AUC, al ser más representativa en escenarios desbalanceados.

Los mejores hiperparámetros encontrados en cada una de las ocho búsquedas se recogen en la Tabla 3.9. Esta configuración fue utilizada en los entrenamientos posteriores mediante la técnica de *underbagging*.

Modelo	Campo	Batch Size	Learning Rate	Optimizador
BETO	about	16	3e-5	adafactor
BETO	description	32	2e-5	adafactor
BETO	motivation	32	3e-5	adafactor
BETO	goal	16	3e-5	schedule_free_radam
RoBERTa	about	32	3e-5	adamw_torch
RoBERTa	description	16	1e-4	schedule_free_radam
RoBERTa	motivation	32	2e-5	adafactor
RoBERTa	goal	16	5e-5	adamw_torch

Tabla 3.9: Hiperparámetros óptimos seleccionados por Optuna.

Estos valores reflejan cómo cada modelo y campo textual requería ajustes específicos para maximizar su rendimiento, confirmando la necesidad de un tratamiento diferenciado por campo en esta tarea.

3.6.2. *Underbagging* y generación de modelos

Cuando se definieron los hiperparámetros óptimos para cada combinación de campo y arquitectura, se procedió al entrenamiento de modelos utilizando la técnica de *underbagging*. Este método consistió en generar subconjuntos aleatorios del conjunto original (con reemplazo) para entrenar modelos independientes, reduciendo la varianza y aumentando la robustez del sistema final.

Modelo	Campo	Bootstrap	Acc.	Prec.	Recall	F1-min	F1-macro	AUC
BETO	about	B1	0.698	0.526	0.534	0.232	0.523	0.612
BETO	about	B2	0.705	0.543	0.541	0.246	0.535	0.615
BETO	about	B3	0.716	0.541	0.569	0.270	0.546	0.621
BETO	description	B1	0.648	0.575	0.619	0.310	0.542	0.649
BETO	description	B2	0.634	0.562	0.612	0.337	0.547	0.647
BETO	description	B3	0.641	0.560	0.619	0.365	0.548	0.651
BETO	motivation	B1	0.689	0.515	0.519	0.212	0.516	0.582
BETO	motivation	B2	0.697	0.530	0.534	0.231	0.522	0.586
BETO	motivation	B3	0.701	0.524	0.540	0.246	0.527	0.589
BETO	goal	B1	0.702	0.579	0.653	0.356	0.590	0.680
BETO	goal	B2	0.705	0.603	0.667	0.392	0.603	0.687
BETO	goal	B3	0.711	0.607	0.672	0.393	0.613	0.687
RoBERTa	about	B1	0.709	0.554	0.580	0.298	0.562	0.642
RoBERTa	about	B2	0.718	0.563	0.578	0.296	0.564	0.646
RoBERTa	about	B3	0.724	0.564	0.589	0.309	0.568	0.648
RoBERTa	description	B1	0.738	0.559	0.570	0.284	0.565	0.646
RoBERTa	description	B2	0.752	0.563	0.576	0.289	0.568	0.648
RoBERTa	description	B3	0.744	0.562	0.571	0.280	0.565	0.650
RoBERTa	motivation	B1	0.709	0.542	0.552	0.262	0.546	0.610
RoBERTa	motivation	B2	0.713	0.548	0.563	0.273	0.553	0.613
RoBERTa	motivation	B3	0.720	0.548	0.564	0.275	0.551	0.614
RoBERTa	goal	B1	0.708	0.579	0.627	0.355	0.582	0.670
RoBERTa	goal	B2	0.722	0.586	0.635	0.364	0.590	0.674
RoBERTa	goal	B3	0.719	0.586	0.649	0.382	0.591	0.673

Tabla 3.10: Resultados individuales por *bootstrap*, campo y arquitectura.

En concreto, se generaron tres *bootstraps* diferentes por campo y por arquitectura, manteniendo siempre la proporción de clases (1:2) establecida para el problema. Esto resultó en un total de 24 modelos entrenados:

- 12 modelos basados en la arquitectura BETO (4 campos $\times$ 3 *bootstraps*).
- 12 modelos basados en RoBERTa-bne (4 campos $\times$ 3 *bootstraps*).

Cada uno de estos modelos se entrenó de forma independiente con los hiperparámetros óptimos obtenidos previamente mediante Optuna. La idea principal detrás del *underbagging* es que los modelos aprendan sobre distintas distribuciones de los

datos, favoreciendo una posterior combinación mediante técnicas de ensemble. La Tabla 3.10 muestra los resultados individuales obtenidos por cada uno de los modelos entrenados sobre los distintos bootstraps, desglosados por campo y arquitectura. Esta información permite observar la variabilidad y estabilidad de los resultados previos al ensemble. Los resultados anteriores evidenciaron una consistencia destacable entre los *bootstraps*, lo que justificó el uso de esta técnica para construir ensambles más estables y robustos.

3.6.3. Resultados del *ensemble* por campo y comparativa de rendimiento

Tras entrenar los tres modelos correspondientes a cada campo mediante *underbagging*, se aplicó una técnica de votación mayoritaria para combinar sus predicciones. Esta fusión permitió la reducción de errores individuales y la mejora de la generalización del sistema. El objeto de este *ensemble* intermedio fue evaluar el potencial clasificatorio de cada campo por separado, de manera independiente a los *ensembles* finales.

Modelo	Campo	Acc.	Prec.	Recall	F1-min	F1-macro	AUC
BETO	about	0.7197	0.5374	0.5479	0.2500	0.5388	0.6208
BETO	description	0.6412	0.5653	0.6167	0.3373	0.5456	0.6509
BETO	motivation	0.7006	0.5231	0.5311	0.2295	0.5219	0.5894
BETO	goal	0.7113	0.5999	0.6638	0.3929	0.6017	0.6874
RoBERTa	about	0.7240	0.5642	0.5889	0.3085	0.5680	0.6476
RoBERTa	description	0.7516	0.5661	0.5778	0.2909	0.5702	0.6495
RoBERTa	motivation	0.7197	0.5482	0.5644	0.2747	0.5505	0.6137
RoBERTa	goal	0.7219	0.5896	0.6371	0.3671	0.5945	0.6743

Tabla 3.11: Resultados del *ensemble* por campo tras fusión de *bootstraps*.

En la Tabla 3.11 se recogen los resultados obtenidos tras aplicar la fusión por campo tanto para BETO como para RoBERTa. Se incluyen métricas como la exactitud (*accuracy*), precisión (*precision*), exhaustividad (*recall*), F1-score de la clase minoritaria (F1-min), F1 macro y AUC. De estos resultados se observó, en primer lugar, que el campo *goal* mostraba un rendimiento significativamente consistente, alcanzando los mejores valores en la mayoría de métricas para ambos modelos. Con RoBERTa se obtenía la mayor *precision* (0,5896), *recall* (0,6371), F1-min (0,3671), F1-macro (0,5945) y AUC (0,6743), indicando que este campo aporta información discriminativa, incluso frente a la clase minoritaria. En el caso de BETO, el campo *goal* también destacaba por encima del resto con una F1-min de 0,3929 y un AUC de 0,6874, véase los valores más elevados de este modelo en toda la comparativa.

A nivel comparativo entre modelos, RoBERTa superaba a BETO en casi todos los campos y métricas, refrendando la hipótesis de que modelos con entrenamiento más reciente y robusto ofrecen mejores capacidades de representación semántica.

Asimismo, el comportamiento más estable de RoBERTa en términos de F1-macro y AUC apuntaba hacia una mayor capacidad de generalización. Por consiguiente, se identificaron las secciones del texto con mayor relevancia para la tarea de predicción, siendo específicamente valioso el campo *goal*, y poniendo de relieve el efecto positivo del uso de arquitecturas Transformer más avanzadas.

3.6.4. Resultados finales

En la última etapa del proceso de modelado se combinaron las predicciones generadas por los *ensembles* de cada campo para cada arquitectura dando lugar a un meta-clasificador como el que puede observarse en la Figura 3.2. La fusión se realizó mediante una votación mayoritaria, sumada a una estrategia basada en la media de las probabilidades (para el cálculo del AUC), con el objetivo de generar una decisión única por instancia.

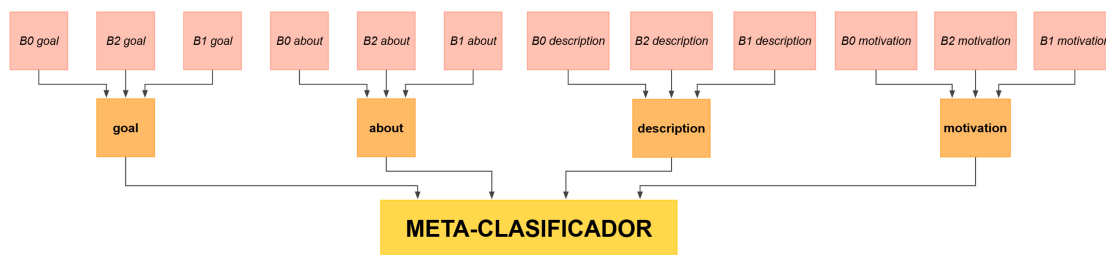


Figura 3.2: Esquema del sistema de clasificación jerárquico con meta-clasificador.

La Tabla 3.12 muestra los resultados obtenidos por cada modelo (BETO y RoBERTa) tras la combinación de los cuatro campos, así como el rendimiento de los modelos individuales de campo ya unificados.

Modelo	Tipo	Acc.	Prec.	Recall	F1-min	F1-macro	AUC
BETO	Ensemble final	0.7346	0.5917	0.6337	0.3655	0.5988	0.6595
RoBERTa	Ensemble final	0.7622	0.5910	0.6116	0.3412	0.5980	0.6759

Tabla 3.12: Resultados finales tras *ensemble* completo por arquitectura.

Ambos modelos alcanzaron un rendimiento competitivo, siendo RoBERTa-base el que lograba la mejor exactitud (76,22 %), así como un AUC destacable frente a BETO. Estas predicciones marcaron el cierre de la fase de entrenamiento y sentaron las bases para el análisis e interpretación en el siguiente epígrafe. Para evaluar el comportamiento de los modelos finales de forma más detallada, se presentan a continuación las matrices de confusión (Figura 3.3 y Figura 3.4) y las curvas ROC (Figura 3.5 y Figura 3.6) correspondientes a cada arquitectura.

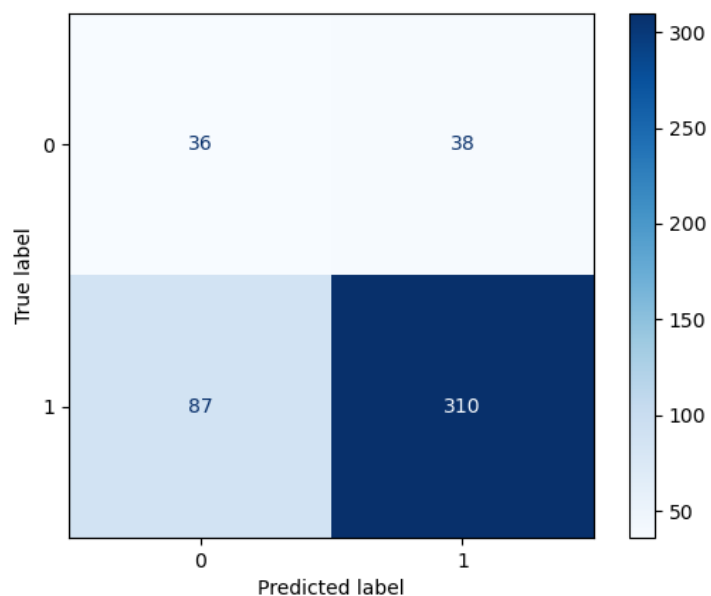


Figura 3.3: Matriz de confusión del modelo final BETO.

Las matrices de confusión de las Figuras 3.3 y 3.4 reflejaron que ambos modelos presentaban un buen desempeño generalizado en la detección de la clase positiva (etiqueta 1), siendo destacable el número de verdaderos positivos alcanzado por RoBERTa (310 frente a 87 falsos negativos), lo que se traduce en una mayor sensibilidad.

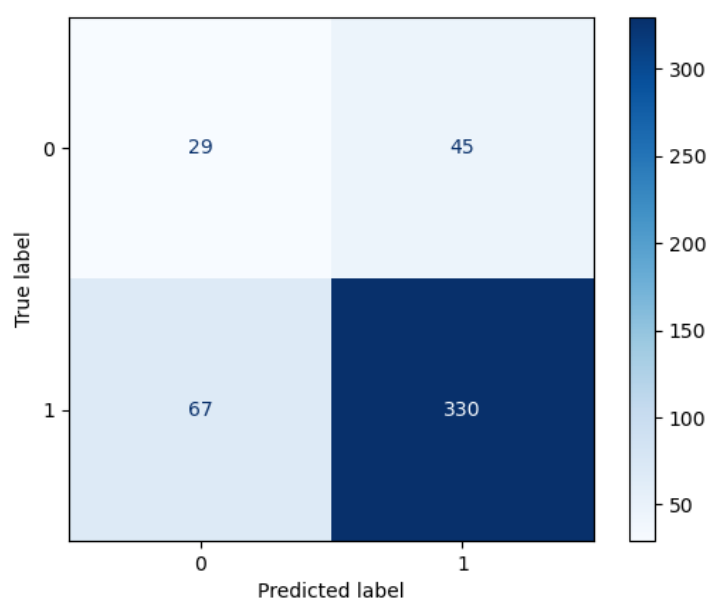


Figura 3.4: Matriz de confusión del modelo final RoBERTa.

Sin embargo, también se observó una tasa relativamente elevada de falsos positivos, lo que sugiere que ambos modelos tienden a sobredimensionar la clase positiva, particularmente en el caso de BETO.

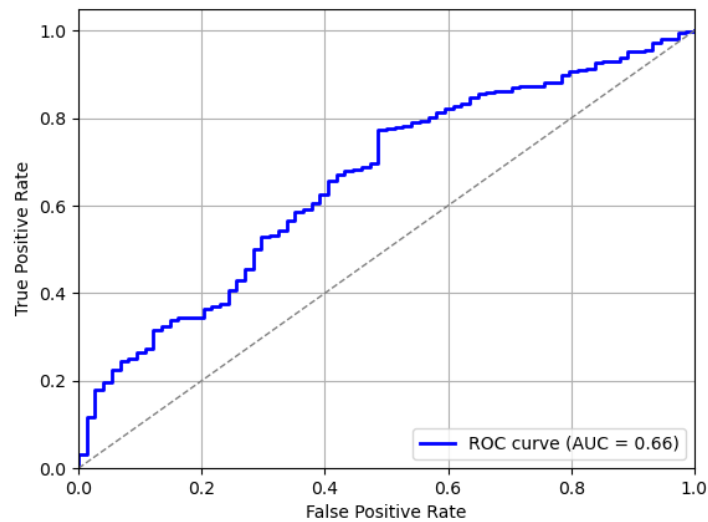


Figura 3.5: Curva ROC del modelo final BETO.

En cuanto a las curvas ROC de las Figuras 3.5 y 3.6, se aprecia una ligera superioridad en la capacidad de discriminación del modelo basado en RoBERTa, con un área bajo la curva (AUC) de 0,68 frente a 0,66 en el caso de BETO. Esta diferencia, aunque modesta, indicaba una mejor capacidad de balancear la tasa de verdaderos positivos y falsos positivos.

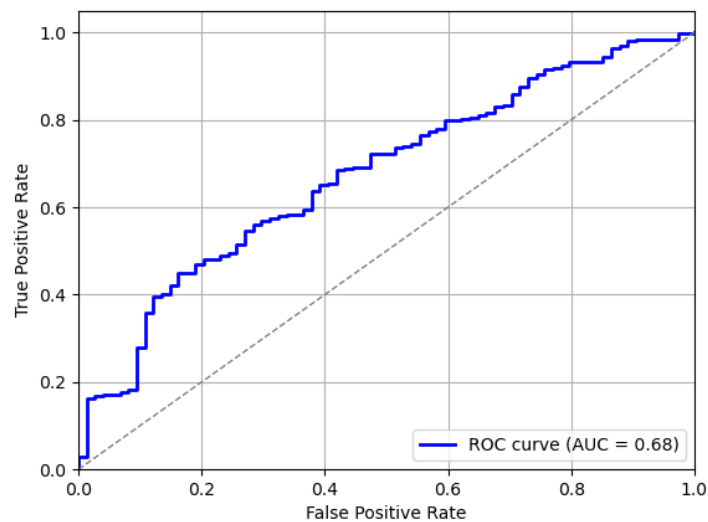


Figura 3.6: Curva ROC del modelo final RoBERTa.

A pesar de que los resultados obtenidos para la clase minoritaria no alcanzan valores elevados, este comportamiento puede explicarse por la complejidad inherente al problema. En primer lugar, las características lingüísticas de los textos, que incluyen descripciones abiertas, lenguaje subjetivo y estructuras sintácticas diversas, dificultan la discriminación precisa de casos menos frecuentes. Además, en tareas de clasificación con clases desbalanceadas, como es este caso, es habitual que los modelos tiendan a favorecer la clase mayoritaria, lo que penaliza las métricas asociadas a la minoritaria.

Por otro lado, esta limitación no es exclusiva de este trabajo. Estudios previos sobre predicción del éxito en campañas de *crowdfunding* como Kickstarter han reportado dificultades similares al aplicar modelos basados en lenguaje natural. Por ejemplo, Chan et al. [59] observaron que, incluso empleando BERT como técnica de representación semántica, la calidad de los textos en secciones específicas afectaba de manera dispar a la capacidad predictiva del modelo, en especial cuando se trataba de variables con menor peso o frecuencia.

Asimismo, otros trabajos como el de Gunduz [61] han mostrado que la precisión en tareas de clasificación binaria puede verse comprometida cuando se introducen factores lingüísticos complejos, afectando especialmente a clases con escasa representación.

3.6.5. Interpretabilidad del modelo

Los resultados globales reflejaron un comportamiento similar en ambas arquitecturas, lo cual es coherente si se considera que ambas se fundamentan en tecnología BERT. Estas similitudes sugieren que las diferencias observadas se deben principalmente a variaciones en el preentrenamiento y no a divergencias estructurales.

Con el objetivo de profundizar en las decisiones de los modelos entrenados y aumentar su transparencia, se aplicó un análisis de interpretabilidad utilizando la herramienta Captum, compatible con modelos de tipo Transformer en PyTorch. Se empleó el método *Integrated Gradients* para calcular la contribución de cada token al resultado final del clasificador.

Como se indica en la Sección 2.9.1 del marco teórico, *Integrated Gradients* es un método de atribución que busca explicar la predicción de un modelo calculando la contribución individual de cada característica de entrada (en este caso, cada token) al resultado final.

El valor resultante para cada token representa su influencia en la predicción: un valor positivo indica que dicho token ha contribuido en el aumento de probabilidad de la clase positiva (en este caso, *éxito*), mientras que un valor negativo sugiere que

ha empujado la predicción hacia la clase negativa (*no éxito*). Los valores cercanos a cero suelen reflejar *tokens* con impacto neutro sobre la decisión del modelo.

Para obtener las gráficas que siguen, se procedió a aplicar *Integrated Gradients* sobre cada muestra del conjunto de test, obteniendo los valores de atribución individuales por *token*. A continuación, se realizó una sumatoria acumulada de los valores de atribución de cada *token* del conjunto de test de todos los proyectos. Esto permitió identificar los *tokens* con mayor influencia global por clase (positiva o negativa), favoreciendo su visualización e interpretación contrastada. Con el fin de aumentar la relevancia de las interpretaciones, antes de generar las visualizaciones se filtraron *Stopwords*, *tokens* con longitud menor a 2 caracteres y signos de puntuación y caracteres no alfabéticos.

Análisis con BETO

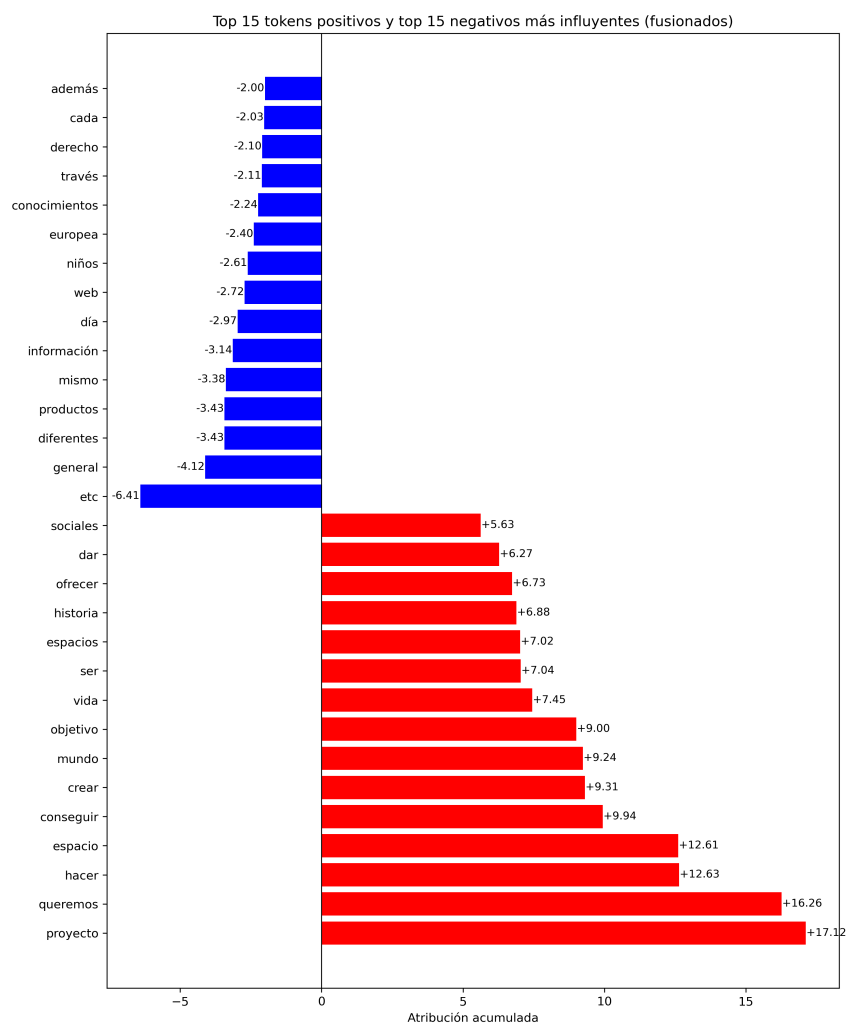


Figura 3.7: *Tokens* más influyentes para la predicción en el campo about (BETO).

En la Figura 3.7 se observa que los *tokens* con mayor influencia positiva son *proyecto*, *queremos*, *hacer*, *espacio* y *crear*. Todos parecen estar vinculados a la acción, el objetivo o la voluntad del equipo promotor, lo cual podría interpretarse por el modelo como señal de confianza y dirección clara.

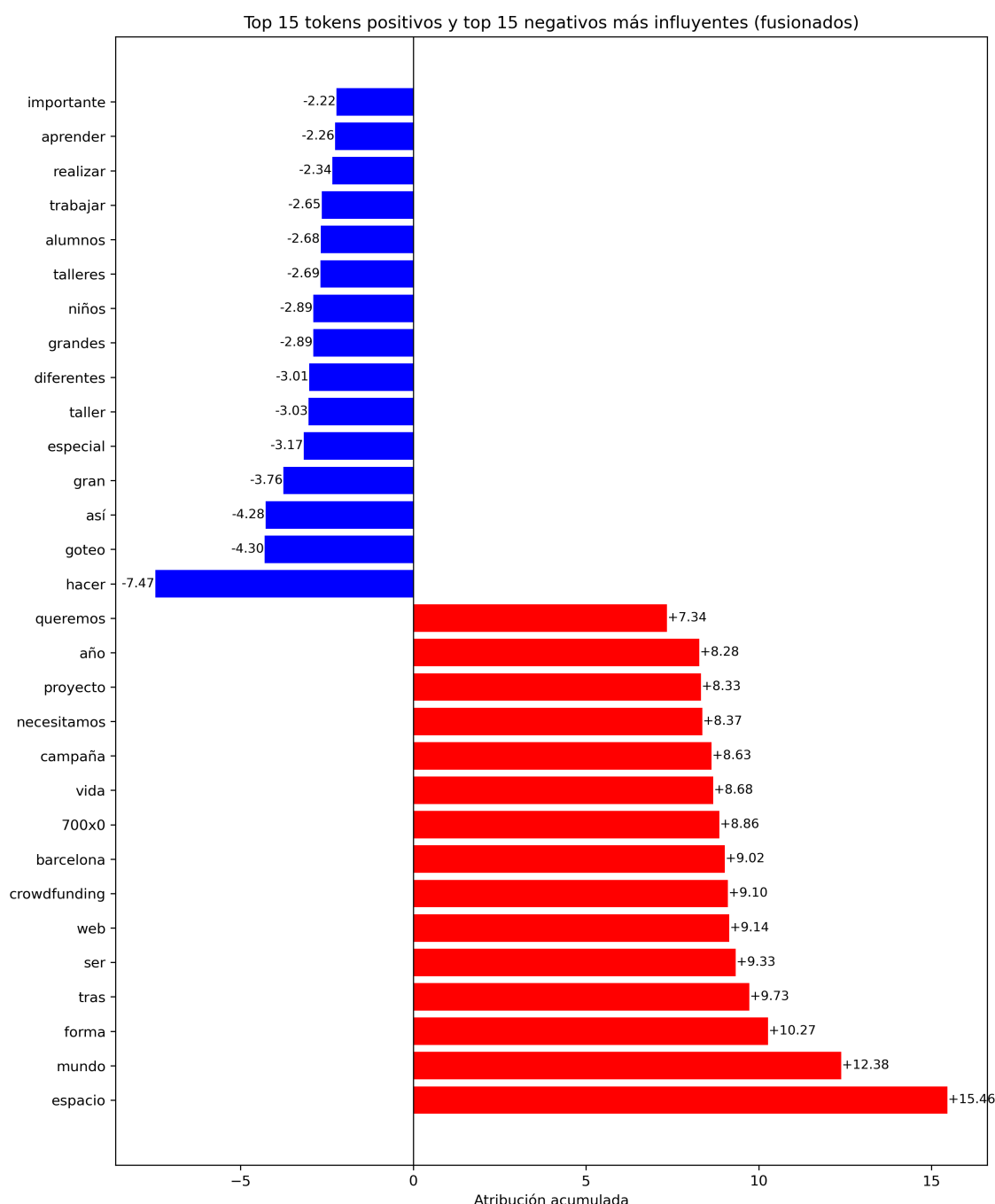


Figura 3.8: *Tokens* más influyentes para la predicción en el campo **description** (BETO).

Por el contrario, los *tokens* negativos más destacados como *además*, *día*, *información*, *general*, *productos* o *etc* reflejan estructuras lingüísticas genéricas o redun-

dantes que el modelo tiende a asociar con descripciones menos convincentes o menos orientadas a la acción.

En el campo *description* (Figura 3.8), el modelo asocia *tokens* como *campaña*, *vida*, *necesitamos*, *comunidad* y *crowdfunding* con una predicción positiva. Estos términos están vinculados a la finalidad social o al enfoque participativo del proyecto, lo que sugiere que el contenido emocional dirigido al colectivo impacta favorablemente.

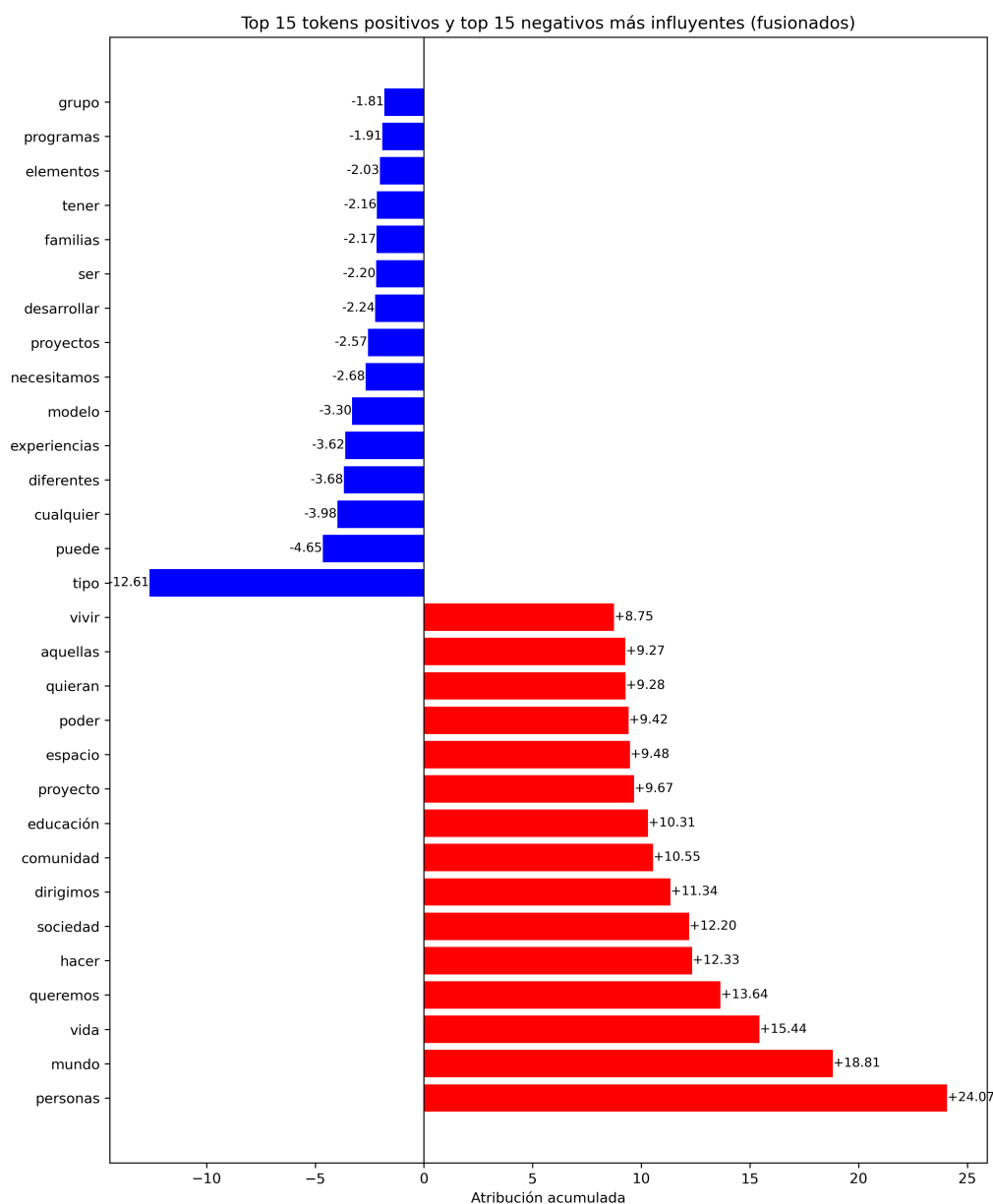


Figura 3.9: *Tokens* más influyentes para la predicción en el campo *motivation* (BETO).

En sentido negativo, se identifican palabras como *importante*, *talleres*, *trabajar* o *especial*, que, si bien no son estrictamente negativas, suelen usarse de forma imprecisa en descripciones más extensas.

La Figura 3.9 muestra que el modelo valora muy positivamente *tokens* ligados al compromiso social y educativo, como *personas*, *vida*, *mundo*, *educación* o *comunidad*. Entre los *tokens* negativos se encuentran *grupo*, *proyectos*, *modelo* o *necesitamos*, lo que puede indicar que el modelo penaliza aquellas motivaciones que se expresan de manera abstracta.

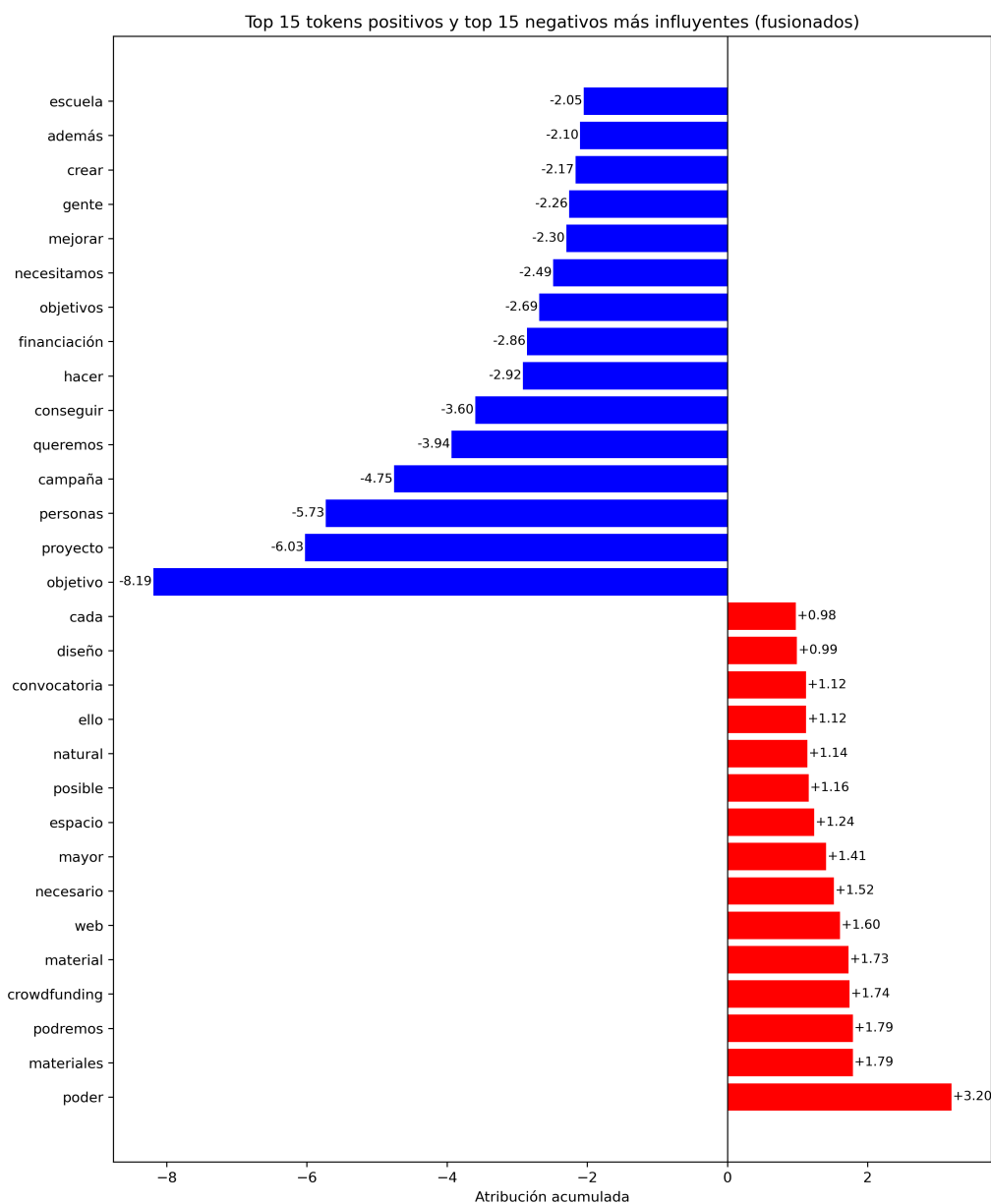


Figura 3.10: *Tokens* más influyentes para la predicción en el campo goal (BETO).

En el campo *goal* (Figura 3.10), las atribuciones acumuladas son menos extremas. Aun así, entre los tokens positivos destacan *posible*, *material*, *crowdfunding* y *poder*, mientras que los negativos más importantes son *objetivo*, *cada*, *diseño* o *campana*. Esto plantea que, al tratarse de un campo más técnico o económico, el modelo

encuentra menor riqueza semántica que en los campos motivacionales. No obstante, algunas palabras clave siguen resultando predictivas.

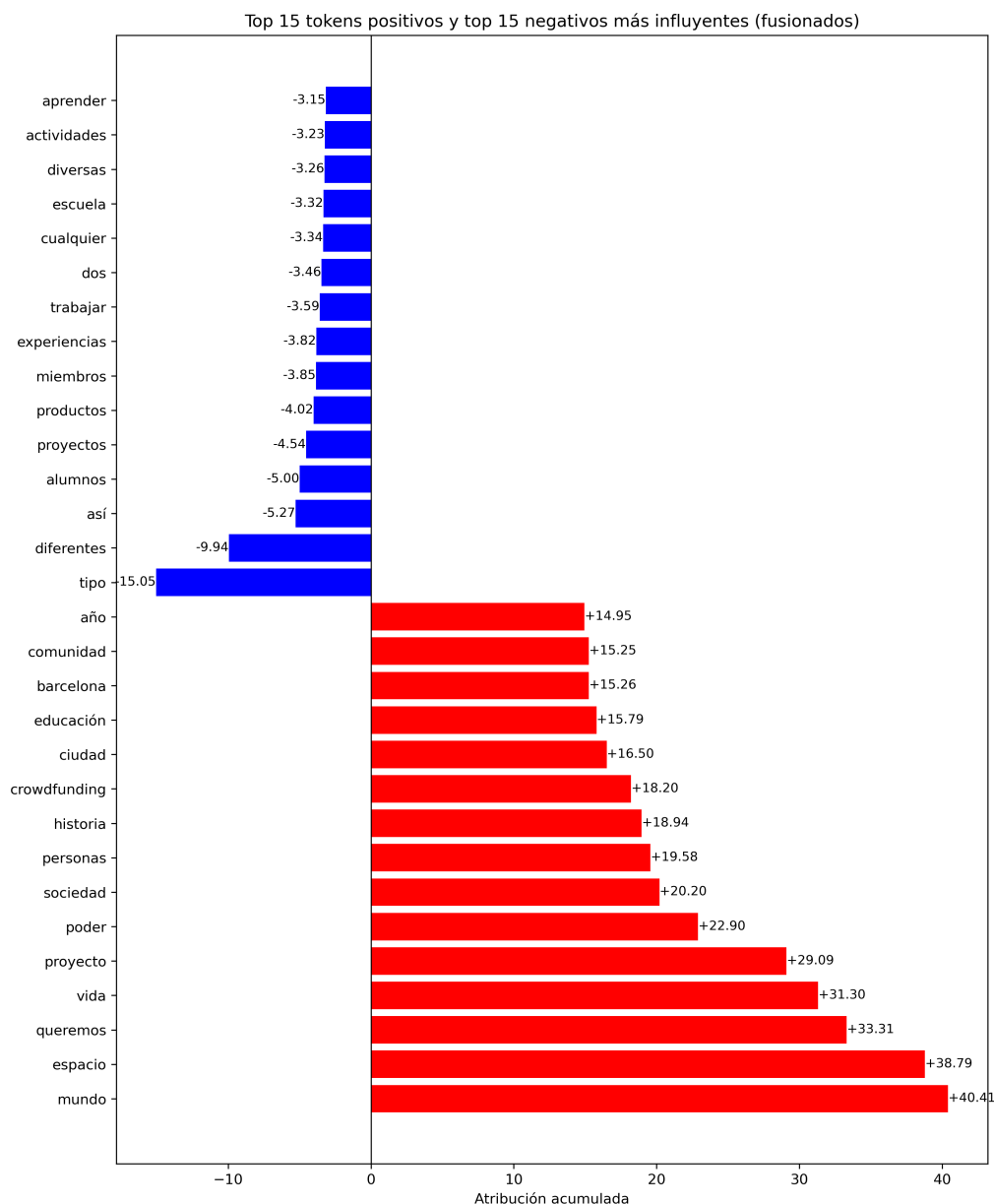


Figura 3.11: *Tokens* más influyentes a nivel global (modelo BETO con campos fusionados).

La Figura 3.11 refuerza todas las observaciones previas. *Tokens* como *proyecto*, *vida*, *queremos*, *mundo* y *espacio* aparecen de forma reiterada para predecir éxito. En el extremo opuesto, términos como *tipo*, *productos*, *alumnos*, *diversas* o *trabajar* aparecen de forma consistente como indicadores de clase negativa. Así, el modelo BETO parece ser fundamentalmente sensible a la presencia de tokens que indican propósito, acción, impacto social y claridad. En contraste, el uso de términos imprecisos, técnicos o que carecen de carga emocional son penalizados.

Análisis con RoBERTa

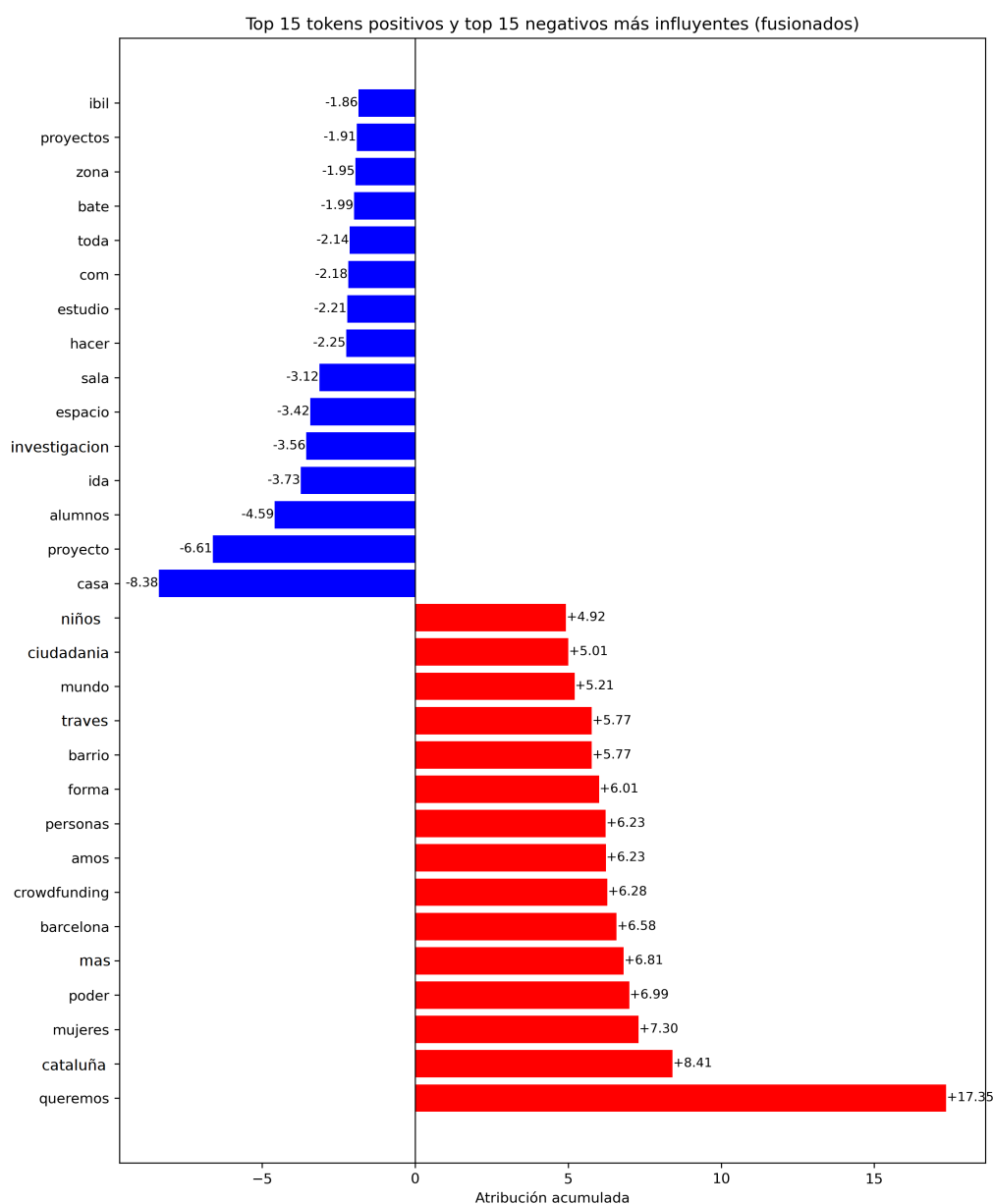


Figura 3.12: *Tokens* más influyentes para la predicción en el campo *about* (RoBERTa).

En el campo *about* (Figura 3.12), destacan como positivos los *tokens* *queremos*, *personas*, *más*, *forma* y *poder*, vinculados a expresiones de intención colectiva, empoderamiento o implicación social. RoBERTa parece interpretar estos términos como indicadores de propósito claro y valor social, elementos que favorecen una predicción positiva. Entre los tokens negativos aparecen *proyecto*, *casa*, *años*, *toda* o *investigación*, los cuales son más frecuentes en formulaciones genéricas o impersonales. El modelo tiende a penalizar textos excesivamente formales, técnicos o desvinculados de una acción inmediata.

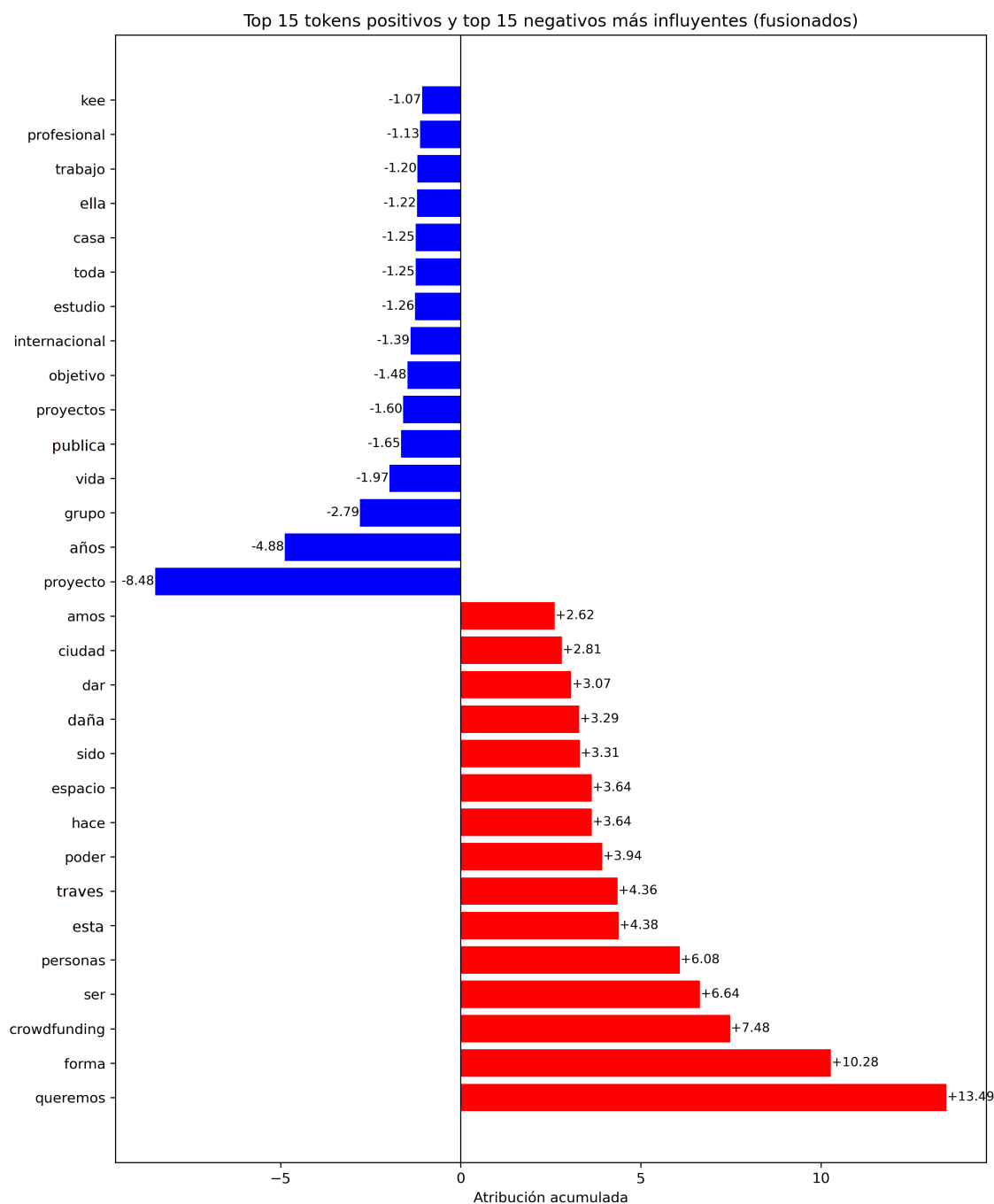


Figura 3.13: *Tokens* más influyentes para la predicción en el campo *description* (RoBERTa).

En el campo *description* (Figura 3.13), las atribuciones son más moderadas. Los *tokens* con mayor contribución positiva son *web*, *poder*, *pretendemos*, *campaña* y *crowdfunding*, reflejando un enfoque orientado a la ejecución del proyecto. Los términos con mayor peso negativo son *proyecto*, *plazo*, *entidades*, *público* o *familias*. Estos términos, si bien podrían considerarse relevantes en algunos contextos, parecen aparecer en descripciones menos convincentes.

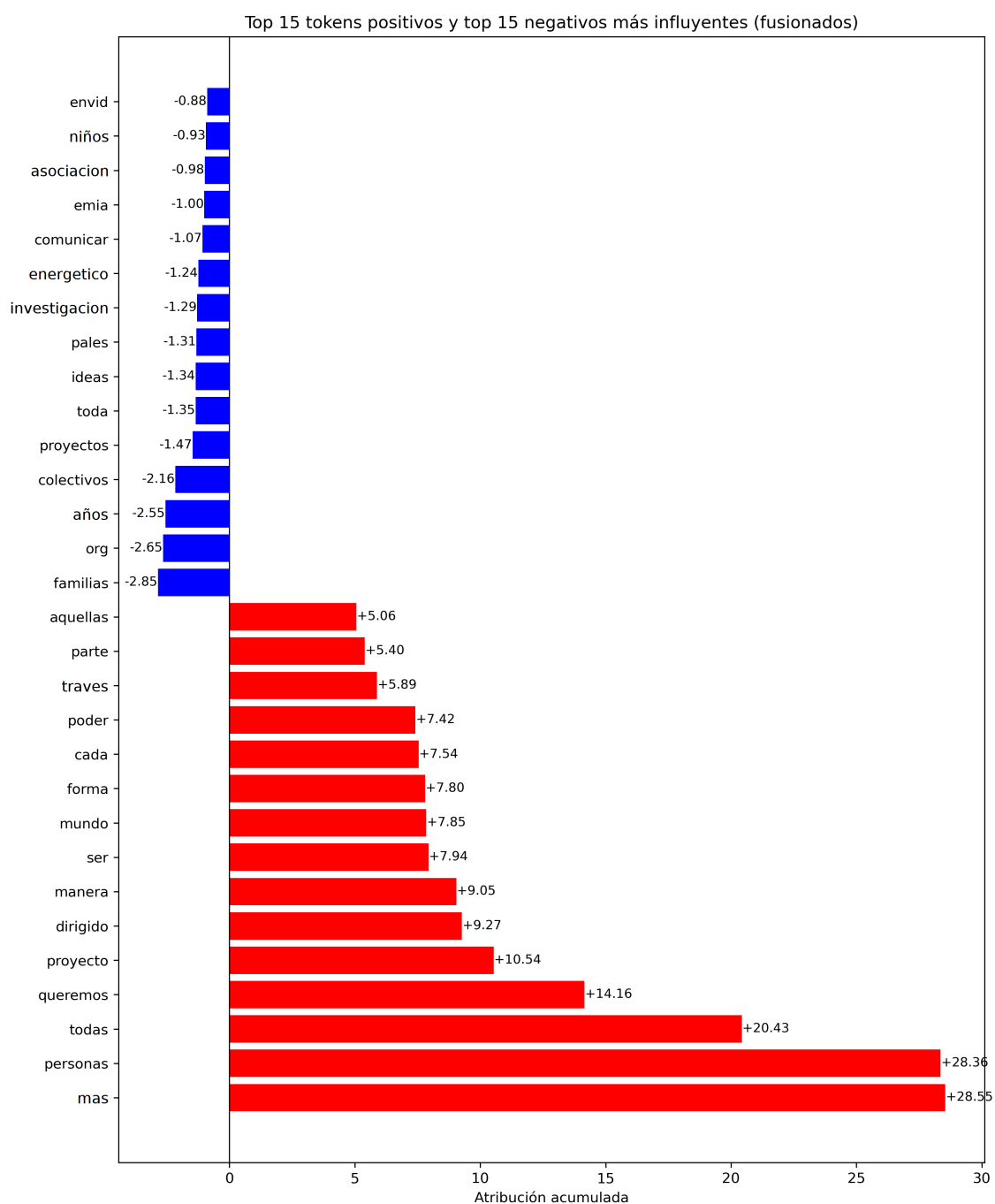


Figura 3.14: *Tokens* más influyentes para la predicción en el campo *motivation* (RoBERTa).

La Figura 3.14, correspondiente al campo *motivation*, muestra *tokens* altamente positivos como *más*, *personas*, *todas*, *queremos* y *proyecto*, alineados con valores colectivos y de transformación social.

Los *tokens* negativos, como *org*, *años*, *colectivos*, *investigación* o *ideas*, reflejan un lenguaje técnico al que el modelo asigna menor probabilidad de éxito.

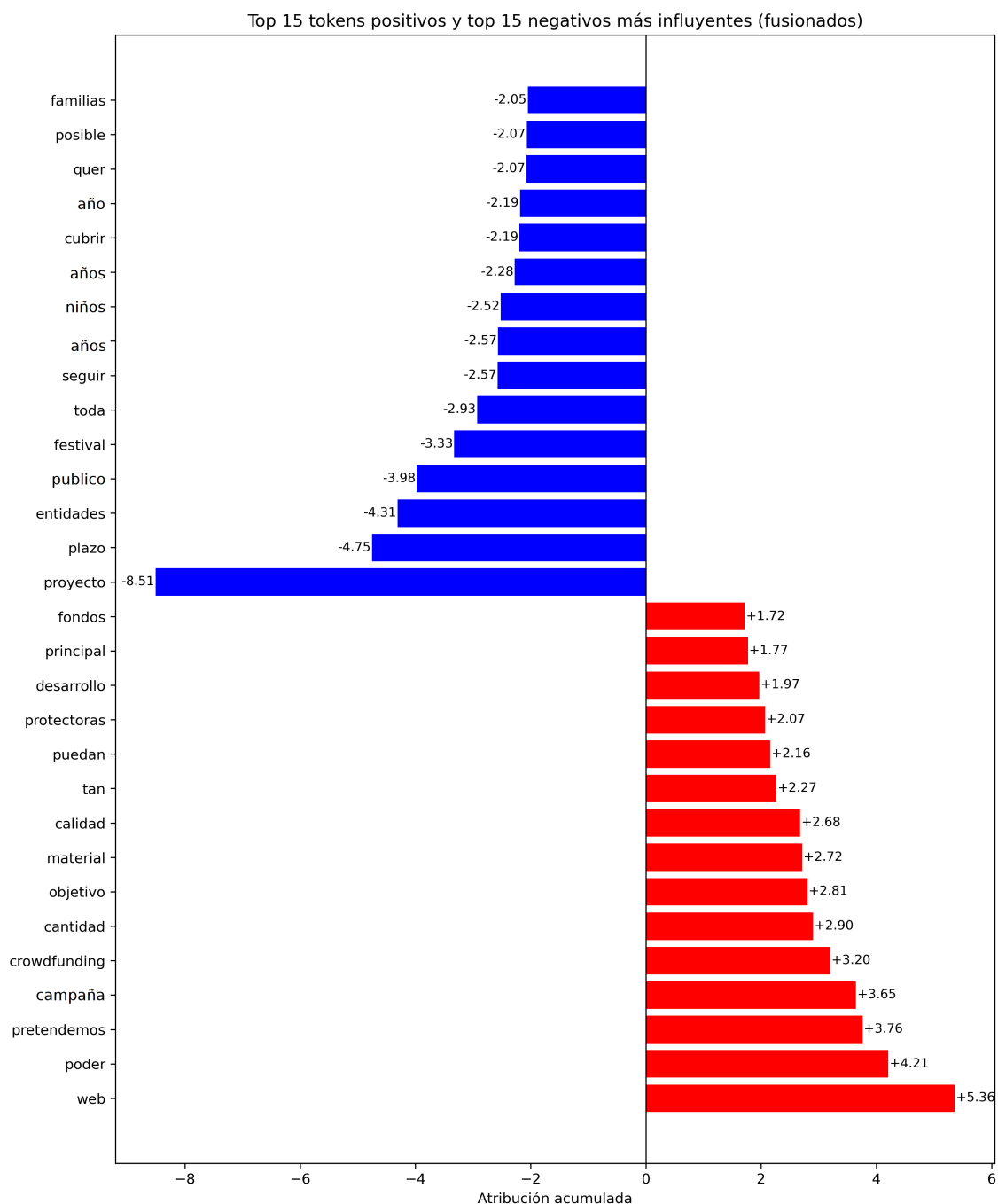


Figura 3.15: *Tokens* más influyentes para la predicción en el campo *goal* (RoBERTa).

En el campo *goal* (Figura 3.15) se repite el patrón de otras secciones: RoBERTa atribuye valor positivo a términos como *queremos*, *forma*, *crowdfunding*, *personas* y *traves*, relacionados con acción colectiva y objetivos alcanzables.

En contraste, palabras como *proyecto*, *años*, *grupo* o *vida* aparecen en la parte negativa, parece, por estar sobreutilizadas en formulaciones descontextualizadas del objetivo económico.

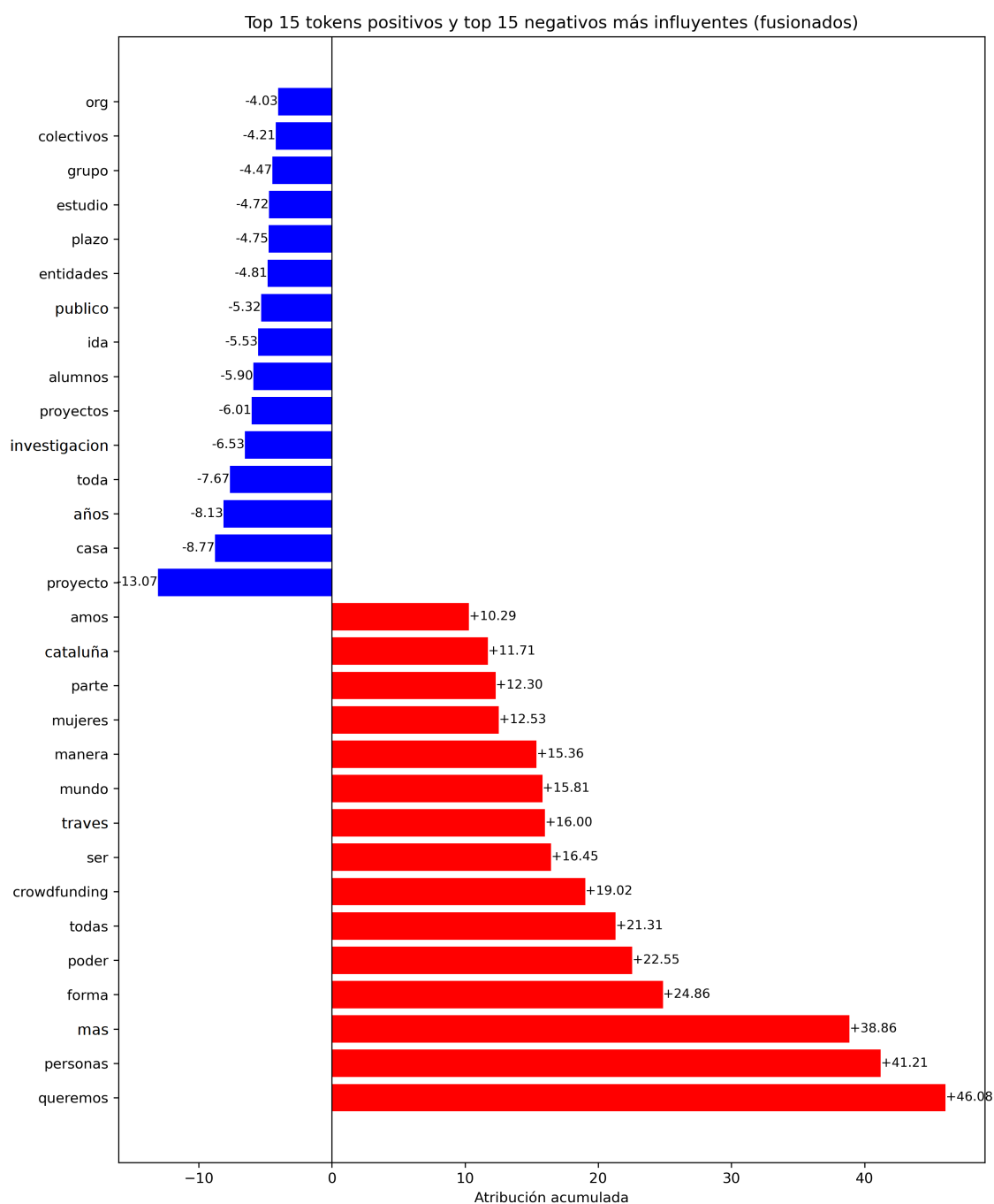


Figura 3.16: *Tokens* más influyentes a nivel global (modelo RoBERTa con campos fusionados).

La Figura 3.16 recoge la atribución acumulada de todos los *tokens* tras fusionar el texto de los campos. Los términos más destacados en predicciones positivas son *queremos*, *personas*, *más*, *forma*, *poder* y *crowdfunding*. Esta lista refleja una fuerte orientación hacia el impacto colectivo y la proactividad del equipo promotor. Los *tokens* más penalizados a nivel global son *proyecto*, *casa*, *años*, *toda*, *investigación* y *alumnos*, de uso repetitivo, lo que puede dificultar al modelo identificar una propues-

ta concreta y diferenciadora. En consecuencia, los análisis de RoBERTa refuerzan las tendencias observadas con BETO, pero con mayor énfasis en términos colectivos y de impacto directo. También, muestra una mayor sensibilidad a matices léxicos y sintácticos, confirmando su solidez en tareas de interpretación contextual.



Figura 3.17: Ejemplo de visualización de atribuciones por *token* mediante *Integrated Gradients*.

La Figura 3.17 muestra una visualización de atribuciones generadas con el método *Integrated Gradients* sobre una predicción del conjunto de test. Cada *token* del texto aparece coloreado en función de su contribución a la salida del modelo: (a) verde si favorece la clase positiva (éxito), (b) rojo si empuja hacia la negativa (no éxito) y (c) blanco si su influencia es neutra. La intensidad cromática está relacionada con el valor de la atribución asignada: cuanto más saturado sea el tono, mayor será la magnitud de la contribución. Este tipo de visualización permite observar, de forma granular, el razonamiento interno del modelo ante un proyecto específico.

En el ejemplo mostrado, se observa que el modelo asigna una atribución positiva destacada a palabras como *solidaridad* o *proyecto*. Estos términos, de corte colectivo, inclusivo o propositivo, aparecen reiteradamente asociados a campañas exitosas y son interpretados como señales de impacto social, objetivo definido o intención transformadora. Su activación positiva refuerza la idea de que el modelo es sensible a estructuras discursivas que apelan a lo comunitario y al bien común, como se señalaba en el análisis previo.

En cambio, expresiones como *estamos* o *buscamos* reciben atribuciones negativas significativas. Aunque estas formas verbales suelen emplearse en contextos proactivos o colaborativos, su uso excesivo o genérico puede diluir el mensaje o proyectar

inseguridad, algo que el modelo detecta como desfavorable. Esta penalización es coherente con el patrón observado en las atribuciones globales, donde el lenguaje impreciso, redundante o centrado en la subjetividad del emisor tiende a correlacionarse con el fracaso.

Además de la atribución individual por *token*, la visualización incluye un valor agregado denominado *Attribution Score*, que representa la suma total de las atribuciones del texto. Este valor ofrece una medida cuantitativa global que complementa la interpretación local por token: cuanto mayor sea su valor (en magnitud y signo), más clara es la tendencia general del texto hacia una clase u otra. De forma coherente con esta interpretación, se observa que los ejemplos correctamente clasificados como exitosos tienden a mostrar *Attribution Scores* positivos, reflejando una suma neta de contribuciones favorables. Por el contrario, los textos asignados a la clase negativa suelen presentar valores negativos, lo que sugiere la presencia de señales lingüísticas más asociadas al fracaso. Esta alineación entre signo del score y clase predicha refuerza su utilidad como indicador de certeza y sesgo direccional de la predicción.

Es importante mencionar también que muchos de los tokens ilustrados no corresponden exactamente a palabras completas del texto original. Esto se debe al proceso de tokenización basado en subpalabras, común en modelos como BERT o RoBERTa. El *tokenizador* introduce marcas como [CLS], [SEP] o fragmentos precedidos por ##, que indican, respectivamente, el inicio de la secuencia, el final y la continuación de una palabra fragmentada. Por ejemplo, la palabra *solidaridad* puede dividirse en *solid* + *##aridad*, dependiendo del vocabulario del modelo. Estas divisiones permiten procesar palabras raras o desconocidas sin perder su significado contextual, aunque pueden afectar ligeramente la legibilidad de la visualización.

Por ende, esta figura ejemplifica cómo el análisis de atribuciones puede visualizar interpretaciones coherentes con los resultados expuestos de los modelos. Esto es, no solo permite confirmar que ciertas palabras o estructuras discursivas tienen mayor influencia en la predicción, sino que también ayuda a detectar sesgos, patrones poco eficaces o zonas de ambigüedad en un contexto léxico.

3.6.6. Contribución del modelo en predicción de proyectos *crowdfunding*

Los hallazgos de atribución semántica obtenidos mediante *Integrated Gradients* presentan que, tanto los modelos BERT como RoBERTa-bne, son sensibles a la carga discursiva de los textos de las campañas de *crowdfunding* en Goteo, favoreciendo términos relacionados a propósito colectivo, acción y carga emocional comunitaria. Esta sensibilidad se mantiene de forma coherente en todos los campos analizados y subraya las señales semánticas predictivas en el lenguaje de los proyectos, incluso

cuando se eliminan fuentes estructuradas de información.

Como se explicaba anteriormente, este enfoque de clasificación parte de una decisión metodológica por la que se decidió no emplear campos que tradicionalmente han sido considerados como predictivos en tareas de *crowdfunding*, tales como el objetivo económico, la duración del proyecto, el número de recompensas, el presupuesto desglosado o las categorías temáticas [59, 60, 61]. Estos atributos, frecuentes en los trabajos relacionados, han sido omitidos con el objetivo de aislar el comportamiento de los modelos de lenguaje sobre datos exclusivamente textuales.

En consecuencia, el sistema desarrollado se enfrenta a la disyuntiva de extraer patrones relevantes a partir del contenido semántico de los textos. Esta predicción lingüística permite estudiar de forma más precisa el papel del lenguaje como variable explicativa del éxito en plataformas de *crowdfunding*.

Además, se realiza sobre la plataforma Goteo, cuya orientación hacia iniciativas con impacto social, educativo o ambiental implica una mayor variabilidad discursiva y libertad narrativa, presentando una estructura textual menos estandarizada que la observada en otras plataformas como *Kickstarter*, orientada a proyectos creativos relacionados con la fotografía, música, videojuegos y otros.

A nivel técnico, esta experimentación implementa un *pipeline* en PyTorch basado en modelos preentrenados monolingües en español (BETO y RoBERTa-bne), optimizados para tareas de clasificación binaria de proyectos sociales. Así, la principal contribución de este enfoque va más allá de su confiabilidad métrica, pues evalúa y diseña un sistema funcional que explora la viabilidad de la predicción semántica pura sobre textos complejos de diversidad lingüística en un dominio exiguamente explorado en lengua española, tal y como se expone en las conclusiones.

3.6.7. Limitaciones

Durante el desarrollo del trabajo se han identificado limitaciones que han condicionado tanto las decisiones metodológicas como la interpretación final de los resultados. Estas, aunque no invalidan los hallazgos obtenidos, sí proporcionan un marco más realista para su evaluación y para el diseño de futuras mejoras.

Una de las principales restricciones estuvo asociada a la infraestructura computacional disponible. Las limitaciones de *hardware*, particularmente en lo relativo a memoria GPU y tiempo de entrenamiento, han obligado a reducir el número de experimentos y a seleccionar modelos cuyo coste computacional fuese asumible. Esto ha supuesto, en algunos casos, renunciar a arquitecturas de mayor tamaño o a estrategias de entrenamiento más exhaustivas como el *fine-tuning* integral de múltiples capas o la búsqueda intensiva de hiperparámetros.

Por otro lado, el propio conjunto de datos presentaba desafíos importantes en cuanto a la variabilidad textual. Los campos analizados mostraban una gran heterogeneidad tanto en longitud como en estilo discursivo. Algunos eran extremadamente breves y escuetos, mientras que otros resultaban extensos o incluían lenguaje ambiguo y poco informativo. Esta diversidad dificultó la extracción de patrones semánticos consistentes, limitando la capacidad de generalización de los modelos.

Asimismo, el marcado desbalanceo de clases ha representado una barrera importante. La presencia mayoritaria de ejemplos etiquetados como proyectos exitosos frente a una clase negativa minoritaria y escasa, ha provocado que muchos modelos tiendan a sesgarse hacia la predicción de la clase dominante. Aunque se evaluaron técnicas de sobre-muestreo para paliar este problema, como la duplicación sintética de ejemplos negativos, finalmente se descartaron al comprobar que introducían ruido semántico y reducían la estabilidad del entrenamiento.

Por último, cabe resaltar que estas condiciones también han influido en la interpretabilidad de los modelos. El uso de técnicas como *Integrated Gradients* ha proporcionado información valiosa, pero la elevada variabilidad lingüística y el bajo número de ejemplos negativos pueden haber limitado la generalización de las explicaciones obtenidas. Por lo que, considerando estas observaciones, se dedujeron las siguientes conclusiones, que apuntan hacia líneas de acción futura en el campo de la predicción del éxito de proyectos *crowdfunding* desde la ingeniería informática.

Conclusiones y trabajo futuro

Este capítulo presenta los principales resultados del proyecto, ofrece una valoración crítica en consecución con los objetivos y propone posibles líneas de mejora y continuidad. Además, integra una estimación del esfuerzo dedicado en cada una de las fases del desarrollo.

4.1. Conclusiones

El presente trabajo desarrolla un sistema de clasificación que permite predecir el éxito o fracaso de campañas de *crowdfunding* a partir del contenido textual de sus descripciones. Este enfoque, centrado principal y exclusivamente en el análisis semántico y discursivo, ha demostrado ser eficaz para modelar patrones lingüísticos asociados al rendimiento de proyectos sociales en la plataforma Goteo, lo cual representa una contribución metodológica, técnica y empírica relevante dentro del PLN en español.

En cumplimiento del primer objetivo específico, se realizó un estudio de las técnicas actuales de clasificación de texto basadas en modelos de lenguaje preentrenados, prestando atención a las aplicaciones más recientes de aprendizaje profundo en tareas de clasificación semántica. Este análisis fundamentó la elección de las arquitecturas BETO y RoBERTa-bne, cumpliendo con el segundo objetivo, que consistía en seleccionar modelos adecuados para el procesamiento de textos en español. Estas dos arquitecturas monolingües se adaptaron de forma natural a las particularidades lingüísticas del corpus analizado y demostraron comportamientos complementarios.

En relación con el tercer objetivo, que buscaba implementar un sistema de entrenamiento ante el desbalanceo de clases, se incorporó la técnica de *underbagging*, con la que se entrenaron múltiples clasificadores independientes sobre subconjuntos equilibrados del conjunto de entrenamiento. Esta estrategia evitó los riesgos asociados al sobre-muestreo en entornos de alta variabilidad semántica, mejorando la estabilidad de los resultados y favoreciendo la capacidad de generalización del modelo.

También, y en respuesta al cuarto objetivo, se llevó a cabo una búsqueda de hiperparámetros mediante Optuna, resultando una herramienta de optimización bayesia-

na que permitió afinar aspectos clave del entrenamiento, como la tasa de aprendizaje o el tamaño de *batch*, y maximizar el rendimiento del sistema sobre un conjunto de validación estratificada.

El quinto objetivo, orientado a la aplicación de técnicas de ensamblado (*ensemble*) para combinar predicciones, se resolvió mediante una integración jerárquica de salidas a nivel de campo (*about*, *description*, *motivation*, y *goal*). Esta combinación multiescalar se mostró eficaz para aumentar las métricas de rendimiento, al captar aspectos semánticos distribuidos entre los campos textuales de cada campaña.

En lo que respecta al sexto objetivo, se evaluó el sistema con métricas estándar como *accuracy*, *F1-score* y *AUC*, obteniendo resultados consistentes, especialmente en la predicción de la clase mayoritaria (*éxito*), lo que valida la pertinencia del *pipeline* propuesto, incluso en condiciones de fuerte desbalanceo de clases y bajo restricciones computacionales.

Uno de los aportes más relevantes fue la resolución del séptimo y último objetivo, que consistía en incorporar interpretabilidad a los modelos entrenados mediante la herramienta Captum. Para ello, se utilizó el método *Integrated Gradients*, que permitió cuantificar la influencia individual de cada token en la predicción, generando visualizaciones a nivel de campo y global. Los datos mostraron que palabras asociadas a propósito colectivo, transformación social o implicación emocional (como *vida*, *mundo*, *comunidad*, *educación*, *queremos* o *proyecto*) tienen un impacto positivo en la predicción de éxito. Por el contrario, términos vagos, neutros o genéricos como *tipo*, *productos*, *trabajar* o *diversas* tienden a correlacionarse negativamente con la clase positiva.

En contraste con estudios previos, [59, 60, 61], que se centraban en plataformas como *Kickstarter* y en campañas en lengua inglesa, y pese a las diferencias metodológicas, se identificaron convergencias léxicas significativas: tanto en inglés (*world*, *help*) como en español (*vida*, *mundo*, *comunidad*), los términos con connotaciones de propósito global o impacto social se relacionaban positivamente con campañas exitosas. De forma análoga, el uso de palabras comunes, poco diferenciadoras o de carácter descriptivo también se vinculaba con campañas fallidas en ambos casos. Este paralelismo sugiere la posible existencia de patrones lingüísticos universales en la percepción del valor discursivo en campañas de *crowdfunding*, y pone de manifiesto el valor añadido del uso de técnicas de atribución basadas en gradientes, que superan las limitaciones del análisis de frecuencia pura.

Un aspecto diferenciador de este trabajo respecto a los señalados fue su aplicación sobre campañas alojadas en Goteo, considerada una plataforma especializada en iniciativas sociales, educativas, medioambientales y culturales, que presenta proyectos de educación comunitaria, soberanía alimentaria, energía renovable, cooperación

vecinal, derechos digitales o innovación cívica abierta. A diferencia de plataformas más comerciales como *Kickstarter*, como se señalaba, orientadas a productos creativos y/o tecnológicos (videojuegos, *gadgets*, cine, diseño), Goteo presenta una mayor libertad discursiva, lo que incrementa el desafío de la modelización automática, pero también enriquece la diversidad léxica y semántica disponible para el análisis.

En este sentido, el proyecto contribuye con un enfoque pionero en lengua española. Mientras que la gran mayoría de estudios sobre PLN y *crowdfunding* se han desarrollado sobre corpus en inglés y estructuras más estandarizadas, este trabajo se ha centrado en campañas redactadas en español, demostrando la viabilidad de los modelos semánticos para tareas de clasificación en contextos lingüísticos menos explorados. Esta línea de investigación puede extenderse hacia el desarrollo de asistentes automáticos de redacción estratégica para campañas en español que sugieran formulaciones más eficaces en función del contenido semántico.

4.2. Trabajo futuro

A partir de los resultados obtenidos y de la estructura metodológica desarrollada, se presentan diversas líneas de trabajo que permitirían extender tanto el alcance como el impacto práctico del sistema propuesto. Estas direcciones incluyen mejoras técnicas, validación externa y aplicaciones orientadas al usuario final, que podrían consolidar la utilidad de los modelos de lenguaje en contextos de innovación social y economía colaborativa en Iberoamérica.

4.2.1. Mejoras técnicas y evolución arquitectónica

Una primera línea de desarrollo consiste en la exploración de modelos de lenguaje de gran escala (LLMs) como *GPT*, *LLaMA* o *Mistral*, entrenados con billones de tokens y capaces de capturar patrones de razonamiento emergente. Mediante técnicas como *instruction tuning*, *few-shot prompting* o *adapter tuning* (LoRA, QLoRA), estos modelos pueden ajustarse a tareas específicas sin necesidad de un entrenamiento intensivo desde cero. Su capacidad para gestionar entradas semánticamente complejas y estructuras narrativas abiertas (como las de Goteo) los convierte en candidatos prometedores para tareas como la clasificación de campañas, el análisis de motivación discursiva o la generación asistida de descripciones eficaces.

Igualmente, estos LLMs podrían ser empleados de forma híbrida, actuando como generadores de representaciones intermedias enriquecidas (*embeddings*) o como asistentes en procesos de anotación automática, validación de ejemplos o incluso en la generación de contrafactuales explicativos. Esta integración escalonada facilitaría el aumento de las capacidades del sistema sin comprometer la eficiencia compu-

tacional, una consideración relevante, dado el entorno de *hardware* limitado que ha condicionado esta experimentación exploratoria.

Otra mejora relevante se relaciona con el tratamiento del desbalanceo de clases, una de las principales limitaciones identificadas en este trabajo. La clase minoritaria (correspondiente a campañas no exitosas) podría ser ampliada mediante nuevas recopilaciones de datos, ya sea mediante *web scraping* de campañas históricas o mediante la colaboración con plataformas aliadas (por ejemplo, *Change.org*). Alternativamente, podrían desarrollarse técnicas de aumento de datos (*data augmentation*) semánticamente controladas, donde los propios LLMs generen versiones negativas de campañas exitosas o reformulen textos con bajo potencial de persuasión, manteniendo la coherencia discursiva y reduciendo el riesgo de sobreajuste.

Asimismo, una línea adicional a considerar sería la incorporación de nuevas fuentes textuales actualmente no consideradas, como comentarios de usuarios, actualizaciones de los promotores o respuestas a preguntas frecuentes. Estos elementos pueden contener señales adicionales sobre la percepción del proyecto, su evolución y la interacción con la comunidad. Factores que, al integrarse, enriquecerían el contexto semántico disponible para el modelo.

4.2.2. Validación externa y generalización

Otro eje prioritario para trabajos futuros es la validación externa del sistema sobre conjuntos de datos ajenos a la plataforma Goteo. Esta extensión permitiría evaluar la capacidad de generalización del modelo en entornos con diferente perfil discursivo, estructura narrativa y propósito comunicativo. Campañas de plataformas como *Verkami*, *Ulule* o incluso una selección anotada de *Kickstarter* traducida al español, podrían utilizarse como banco de pruebas alternativo para comprobar la robustez del sistema fuera del dominio de entrenamiento original.

Esta validación cruzada también permitiría afinar hipótesis sobre los patrones lingüísticos universales detectados en la comparación con trabajos relacionados, y evaluar si la sensibilidad semántica observada en los modelos entrenados se mantiene estable ante variaciones temáticas o culturales.

4.2.3. Aplicaciones prácticas y transferencia de conocimiento

Más allá del ámbito académico, los resultados de este trabajo pueden traducirse en herramientas de asistencia práctica para promotores de campañas, especialmente en entornos sociales, cooperativos o comunitarios donde el impacto comunicativo es crítico. A partir del *pipeline* actual, es posible concebir una interfaz web o un módulo integrado en plataformas de *crowdfunding* que permita a los usuarios evaluar auto-

máticamente la claridad, efectividad y persuasión de sus textos, sugiriendo mejoras o destacando posibles puntos débiles.

Estas herramientas podrían incorporar interpretaciones tokenizadas generadas por métodos como *Integrated Gradients*, proporcionando a los usuarios una visualización intuitiva de las partes del texto que contribuyen positiva o negativamente a la percepción del éxito. En este sentido, se abre una línea de desarrollo hacia asistentes inteligentes para la redacción estratégica en español, especialmente valiosos en sectores con menor tradición digital o capacidad técnica.

En la misma línea, la base metodológica desarrollada en este trabajo podría aplicarse a otros ámbitos en los que el lenguaje abierto y narrativo tenga un peso mayor, como solicitudes de subvención, programas de impacto social, proyectos de innovación educativa o convocatorias culturales. La transferencia de este conocimiento contribuiría a democratizar el acceso a herramientas de PLN aplicadas, generando valor en comunidades y contextos donde el lenguaje sigue siendo el principal vehículo de legitimidad e impacto.

4.3. Planificación temporal del trabajo realizado

En el desarrollo del presente TFG se han invertido aproximadamente:

- **70 horas** en estudio teórico y comprensión de modelos Transformers, técnicas de PLN y evaluación de clasificadores.
- **80 horas** en aprendizaje práctico de tecnologías y librerías necesarias (Transformers, Hugging Face, PyTorch, Optuna, Captum).
- **100 horas** en procesamiento, preparación de datos y entrenamiento de modelos.
- **60 horas** en interpretación de resultados, generación de gráficos y análisis comparativos.
- **40 horas** en redacción de la memoria y preparación de los recursos finales.

El trabajo ha supuesto un reto considerable tanto a nivel técnico como organizativo, permitiendo afianzar conocimientos avanzados de procesamiento del lenguaje natural y aprendizaje profundo en un contexto real y desde un enfoque pragmático.

Toda la implementación desarrollada para este proyecto, incluyendo los *scripts* de preprocesamiento, configuración de modelos y análisis de interpretabilidad, se encuentra disponible en el [A.2](#) para su consulta y reutilización en futuras experimentaciones.

Referencias

- [1] Stuart J. Russell y Peter Norvig. *Artificial Intelligence: A Modern Approach*. 3rd. Prentice Hall, 2010.
- [2] Justin Grimmer, Margaret E. Roberts y Brandon M. Stewart. *Text as Data: A New Framework for Machine Learning and the Social Sciences*. Princeton University Press, 2022.
- [3] Frank Rosenblatt. “The perceptron: a probabilistic model for information storage and organization in the brain”. En: *Psychological review* 65.6 (1958), pág. 386.
- [4] Marvin Minsky y Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, 1969.
- [5] Ian Goodfellow, Yoshua Bengio y Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [6] Christopher M Bishop. *Neural Networks for Pattern Recognition*. Oxford University Press, 1995.
- [7] David E Rumelhart, Geoffrey E Hinton y Ronald J Williams. “Learning representations by back-propagating errors”. En: *Nature* 323.6088 (1986), págs. 533-536.
- [8] Vinod Nair y Geoffrey E Hinton. “Rectified linear units improve restricted boltzmann machines”. En: (2010), págs. 807-814.
- [9] Sepp Hochreiter. “Untersuchungen zu dynamischen neuronalen Netzen”. Tesis doct. Technische Universität München, 1991.
- [10] Ashish Vaswani, Noam Shazeer, Niki Parmar et al. “Attention is all you need”. En: *Advances in Neural Information Processing Systems* 30 (2017).
- [11] Nitish Shirish Keskar et al. “On large-batch training for deep learning: Generalization gap and sharp minima”. En: *arXiv preprint arXiv:1609.04836* (2017).
- [12] Ilya Loshchilov y Frank Hutter. “Decoupled weight decay regularization”. En: *International Conference on Learning Representations (ICLR)* (2019).
- [13] Nitish Srivastava et al. “Dropout: A simple way to prevent neural networks from overfitting”. En: *Journal of Machine Learning Research* 15.1 (2014), págs. 1929-1958.
- [14] Lutz Prechelt. “Early stopping—But when?” En: *Neural Networks: Tricks of the Trade*. Springer, 1998, págs. 55-69.
- [15] Christopher D Manning e Hinrich Schütze. *Foundations of Statistical Natural Language Processing*. MIT press, 1999.
- [16] Tomas Mikolov et al. “Efficient estimation of word representations in vector space”. En: *arXiv preprint arXiv:1301.3781* (2013).
- [17] Jeffrey Pennington, Richard Socher y Christopher D Manning. “Glove: Global vectors for word representation”. En: *EMNLP*. 2014, págs. 1532-1543.
- [18] Piotr Bojanowski et al. “Enriching Word Vectors with Subword Information”. En: *Transactions of the Association for Computational Linguistics* 5 (2017), págs. 135-146.
- [19] Jacob Devlin et al. “BERT: Pre-training of deep bidirectional transformers for language understanding”. En: *NAACL* (2019), págs. 4171-4186.

- [20] José Cañete et al. *Spanish Pre-Trained BERT Model and Evaluation Data*. <https://github.com/dccuchile/beto>. Proceedings of the Practical ML for Developing Countries Workshop at ICLR 2020. 2020.
- [21] Alec Radford et al. “Language models are unsupervised multitask learners”. En: *OpenAI Blog* (2019).
- [22] Colin Raffel et al. “Exploring the limits of transfer learning with a unified text-to-text transformer”. En: *JMLR* 21.140 (2020), págs. 1-67.
- [23] Sepp Hochreiter y Jürgen Schmidhuber. “Long short-term memory”. En: *Neural computation* 9.8 (1997), págs. 1735-1780.
- [24] Dzmitry Bahdanau, Kyunghyun Cho y Yoshua Bengio. “Neural machine translation by jointly learning to align and translate”. En: *arXiv preprint arXiv:1409.0473* (2015).
- [25] Thang Luong, Hieu Pham y Christopher D Manning. “Effective approaches to attention-based neural machine translation”. En: *arXiv preprint arXiv:1508.04025* (2015).
- [26] Matthew E Peters et al. “Deep contextualized word representations”. En: *NAACL-HLT*. 2018, págs. 2227-2237.
- [27] Jimmy Lei Ba, Jamie Ryan Kiros y Geoffrey E Hinton. “Layer normalization”. En: *arXiv preprint arXiv:1607.06450* (2016).
- [28] Yi Tay et al. “Efficient transformers: A survey”. En: *arXiv preprint arXiv:2009.06732* (2020).
- [29] Emma Strubell, Ananya Ganesh y Andrew McCallum. “Energy and policy considerations for deep learning in NLP”. En: *ACL* (2019), págs. 3645-3650.
- [30] Sinong Wang et al. “Linformer: Self-attention with linear complexity”. En: *arXiv preprint arXiv:2006.04768* (2020).
- [31] Iz Beltagy, Matthew E Peters y Arman Cohan. “Longformer: The long-document transformer”. En: *arXiv preprint arXiv:2004.05150* (2020).
- [32] Krzysztof Choromanski et al. “Rethinking attention with performers”. En: *ICLR* (2021).
- [33] Yinhan Liu et al. “RoBERTa: A Robustly Optimized BERT Pretraining Approach”. En: *arXiv preprint arXiv:1907.11692* (2019).
- [34] Rodrigo Agerri, Jon Campos y Aitor González-Agirre. “RoBERTa-base-bne and other transformer-based models for Spanish: Overview of the RTE and NLI tasks at IberLEF 2020”. En: *CEUR Workshop Proceedings*. 2020.
- [35] Cristian Gonzalez-Gallardo, Aitor Gonzalez-Agirre y Rodrigo Agerri. “RoBERTuito: a pre-trained language model for Spanish based on RoBERTa”. En: *arXiv preprint arXiv:2308.02976* (2023).
- [36] Karl Weiss, Taghi M. Khoshgoftaar y DingDing Wang. “A survey of transfer learning”. En: *Journal of Big Data* 3.1 (2016), págs. 1-40.
- [37] Lisa Torrey y Jude Shavlik. “Transfer learning”. En: *Handbook of Research on Machine Learning Applications and Trends: Algorithms, Methods, and Techniques*. IGI Global, 2010, págs. 242-264.

- [38] Jeremy Howard y Sebastian Ruder. “Universal Language Model Fine-tuning for Text Classification”. En: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2018, págs. 328-339.
- [39] Alexandra Chronopoulou, Kasper Stypulkowski y Nikolaos Aletras. “An Embarrassingly Simple Approach for Transfer Learning from Pretrained Language Models”. En: *arXiv preprint arXiv:1902.10547* (2019).
- [40] Marius Mosbach, Maksym Andriushchenko y Dietrich Klakow. “On the Stability of Fine-tuning BERT: Misconceptions, Explanations, and Strong Baselines”. En: *International Conference on Learning Representations (ICLR)* (2021).
- [41] Thomas G. Dietterich. “Ensemble methods in machine learning”. En: *Multiple classifier systems*. Springer, 2000, págs. 1-15.
- [42] David Opitz y Richard Maclin. “Popular ensemble methods: An empirical study”. En: *Journal of Artificial Intelligence Research* 11 (1999), págs. 169-198.
- [43] Ludmila I. Kuncheva. *Combining Pattern Classifiers: Methods and Algorithms*. Wiley-Interscience, 2004.
- [44] Robi Polikar. “Ensemble based systems in decision making”. En: *IEEE Circuits and Systems Magazine* 6.3 (2006), págs. 21-45.
- [45] Alberto Fernández et al. *Learning from imbalanced data sets*. Springer, 2018.
- [46] Tomasz Maciejewski y Jerzy Stefanowski. “Local neighbourhood extension of SMO-TE for mining imbalanced data”. En: *2011 IEEE Symposium on Computational Intelligence and Data Mining (CIDM)*. IEEE. 2011, págs. 104-111.
- [47] Tianqi Chen y Carlos Guestrin. “XGBoost: A scalable tree boosting system”. En: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016, págs. 785-794.
- [48] David H. Wolpert. “Stacked generalization”. En: *Neural Networks* 5.2 (1992), págs. 241-259.
- [49] Marco Tulio Ribeiro, Sameer Singh y Carlos Guestrin. “Why Should I Trust You? Explaining the Predictions of Any Classifier”. En: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM. 2016, págs. 1135-1144.
- [50] Scott M Lundberg y Su-In Lee. “A unified approach to interpreting model predictions”. En: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.
- [51] Mukund Sundararajan, Ankur Taly y Qiqi Yan. “Axiomatic attribution for deep networks”. En: *Proceedings of the 34th International Conference on Machine Learning*. PMLR. 2017, págs. 3319-3328.
- [52] Artur Kokhlikyan et al. “Captum: A unified and generic model interpretability library for PyTorch”. En: *arXiv preprint arXiv:2009.07896* (2020).
- [53] Hao Yin, Jiawei Li y Tong Liu. “Evaluating Transformer Interpretability with Captum in Sentiment Analysis Tasks”. En: *arXiv preprint arXiv:2204.06753* (2022).

- [54] Marina Sokolova y Guy Lapalme. “A systematic analysis of performance measures for classification tasks”. En: *Information Processing & Management* 45.4 (2009), págs. 427-437.
- [55] Christopher D. Manning, Prabhakar Raghavan e Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [56] Takaya Saito y Marc Rehmsmeier. “The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets”. En: *PLOS ONE* 10.3 (2015), e0118432.
- [57] Laszlo A. Jeni, Jeffrey F. Cohn y Fernando De La Torre. “Facing imbalanced data—Recommendations for the use of performance metrics”. En: *2013 Humaine Association Conference on Affective Computing and Intelligent Interaction*. IEEE. 2013, págs. 245-251.
- [58] Tom Fawcett. “An introduction to ROC analysis”. En: *Pattern Recognition Letters* 27.8 (2006), págs. 861-874.
- [59] C.S. Richard Chan, Charuta Pethe y Steven Skiena. “Natural language processing versus rule-based text analysis: Comparing BERT score and readability indices to predict crowdfunding outcomes”. En: *Journal of Business Venturing Insights* 16 (2021), e00276. DOI: [10.1016/j.jbvi.2021.e00276](https://doi.org/10.1016/j.jbvi.2021.e00276).
- [60] Zijian Zhang y Raymond Y.K. Lau. “Exploiting Multimodal Features and Deep Learning for Predicting Crowdfunding Successes”. En: *2024 IEEE International Conference on Omni-layer Intelligent Systems (COINS)*. IEEE. 2024. DOI: [10.1109/COINS61597.2024.10622132](https://doi.org/10.1109/COINS61597.2024.10622132).
- [61] Hakan Gunduz. “Comparative analysis of BERT and FastText representations on crowdfunding campaign success prediction”. En: *PeerJ Computer Science* 10 (2024), e2316. DOI: [10.7717/peerj-cs.2316](https://doi.org/10.7717/peerj-cs.2316).

A.1. Tablas de resultado de búsqueda hiperparámetros con Optuna

En este anexo se presentan los resultados obtenidos durante el proceso de búsqueda de hiperparámetros mediante Optuna. Estas tablas recogen las métricas de evaluación obtenidas por los diferentes modelos entrenados en cada una de las configuraciones evaluadas. Los resultados se agrupan por archivo.

AUC	f1-macro	precision	batch_size	optimizer	f1_mayoritaria	recall	weight_decay	f1_minoritaria	accuracy	learning_rate
0.579805	0.488770	0.492174	32	schedule_free_radam	0.874092	0.495200	0.001	0.103448	0.779193	0.000030
0.604840	0.563589	0.562627	16	adafactor	0.811908	0.594050	0.100	0.315271	0.704883	0.000010
0.602236	0.513597	0.529051	32	schedule_free_radam	0.884337	0.516271	0.100	0.142857	0.796178	0.000030
0.637518	0.557018	0.555549	16	adafactor	0.816600	0.580077	0.001	0.297436	0.709130	0.000030
0.600432	0.490748	0.517460	32	schedule_free_radam	0.897285	0.505616	0.010	0.084211	0.815287	0.000020
0.583719	0.531534	0.531375	32	adafactor	0.819974	0.541596	0.001	0.243094	0.709130	0.000030
0.610865	0.534877	0.533377	16	adafactor	0.838046	0.538958	0.100	0.231707	0.732484	0.000100
0.591412	0.542581	0.543320	32	adafactor	0.809651	0.562785	0.010	0.275510	0.698514	0.000010
0.616158	0.487276	0.556156	32	adafactor	0.650564	0.605759	0.100	0.323988	0.539278	0.000020
0.591344	0.557331	0.554255	32	schedule_free_radam	0.828947	0.572401	0.001	0.285714	0.723992	0.000100
0.614644	0.548248	0.546771	16	adamw_torch	0.851334	0.550293	0.001	0.245161	0.751592	0.000005
0.614950	0.535920	0.544029	16	adafactor	0.782247	0.571380	0.100	0.289593	0.666667	0.000050
0.566717	0.510747	0.514885	16	adamw_torch	0.801598	0.520985	0.001	0.219895	0.683652	0.000020
0.604091	0.526244	0.542335	16	adafactor	0.759602	0.572759	0.100	0.292887	0.641189	0.000020
0.589676	0.527342	0.528499	32	adafactor	0.857500	0.526585	0.010	0.197183	0.757962	0.000005

Tabla A.1: Tabla 1: Métricas e Hiperparámetros BETO *about*.

AUC	f1-macro	precision	batch_size	optimizer	f1_mayoritaria	recall	weight_decay	f1_minoritaria	accuracy	learning_rate
0.623221	0.574191	0.569062	16	schedule_free_radam	0.837270	0.590952	0.001	0.311111	0.736730	0.000030
0.605998	0.573073	0.568637	16	adamw_torch	0.828685	0.595650	0.010	0.317460	0.726115	0.000005
0.625298	0.558871	0.558412	16	adamw_torch	0.810811	0.587293	0.010	0.306931	0.702760	0.000005
0.631987	0.561908	0.559914	32	schedule_free_radam	0.817694	0.586834	0.001	0.306122	0.711253	0.000100
0.625026	0.569851	0.566058	16	schedule_free_radam	0.825566	0.593131	0.100	0.314136	0.721868	0.000050
0.610167	0.572582	0.567992	32	adafactor	0.847545	0.582017	0.100	0.297619	0.749469	0.000005
0.626659	0.564559	0.560933	16	schedule_free_radam	0.828042	0.583396	0.001	0.301075	0.723992	0.000050
0.612771	0.519062	0.520983	16	adafactor	0.811096	0.528542	0.100	0.227027	0.696391	0.000100
0.617214	0.578059	0.576867	16	adafactor	0.864728	0.579379	0.010	0.291391	0.772824	0.000020
0.635322	0.546456	0.561169	32	adafactor	0.765043	0.606542	0.010	0.327869	0.651805	0.000020
0.634880	0.567493	0.569010	32	adafactor	0.801653	0.609742	0.010	0.333333	0.694268	0.000020
0.637092	0.563393	0.568574	32	adafactor	0.790503	0.613180	0.010	0.336283	0.681529	0.000020
0.634880	0.567493	0.569010	32	adafactor	0.801653	0.609742	0.010	0.333333	0.694268	0.000020
0.628991	0.579692	0.574389	32	adafactor	0.831341	0.603666	0.010	0.328042	0.730361	0.000010
0.634880	0.567493	0.569010	32	adafactor	0.801653	0.609742	0.010	0.333333	0.694268	0.000020

Tabla A.2: Tabla 2: Métricas y Hiperparámetros BETO *description*.

AUC	fl-macro	precision	batch_size	optimizer	fl_mayoritaria	recall	weight_decay	fl_minoritaria	accuracy	learning_rate
0.529308	0.457373	0.421444	16	adamw_torch	0.914747	0.500000	0.010	0.000000	0.842887	0.000100
0.641722	0.565728	0.561608	32	adafactor	0.833114	0.581677	0.100	0.298343	0.730361	0.000100
0.633535	0.563284	0.560372	16	adafactor	0.822903	0.585115	0.010	0.303665	0.717622	0.000050
0.661328	0.555501	0.563694	32	adafactor	0.781997	0.606883	0.001	0.329004	0.670913	0.000030
0.650129	0.549765	0.558917	32	adamw_torch	0.779184	0.598866	0.010	0.320346	0.666667	0.000020
0.622200	0.532803	0.531861	16	schedule_free_radam	0.846252	0.534260	0.001	0.219355	0.743100	0.000020
0.621571	0.568547	0.564472	32	adamw_torch	0.847938	0.576520	0.100	0.289157	0.749469	0.000100
0.523929	0.457373	0.421444	16	schedule_free_radam	0.914747	0.500000	0.010	0.000000	0.842887	0.000005
0.633842	0.552569	0.553048	16	adamw_torch	0.808108	0.579277	0.010	0.297030	0.698514	0.000005
0.584553	0.540043	0.542725	16	schedule_free_radam	0.864259	0.538379	0.010	0.215827	0.768577	0.000020
0.613061	0.566938	0.562895	32	adafactor	0.846452	0.575260	0.001	0.287425	0.747346	0.000030
0.613367	0.523257	0.545308	32	adamw_torch	0.744928	0.580434	0.001	0.301587	0.626327	0.000010
0.646283	0.546200	0.567105	32	adafactor	0.887273	0.542038	0.001	0.205128	0.802548	0.000030
0.625894	0.566633	0.574893	32	adafactor	0.877627	0.561968	0.001	0.255639	0.789809	0.000030
0.648938	0.533363	0.547867	32	adamw_torch	0.764205	0.582034	0.100	0.302521	0.647558	0.000020

Tabla A.3: Tabla 3: Métricas y Hiperparámetros BETO *motivation*).

AUC	fl-macro	precision	batch_size	optimizer	fl_mayoritaria	recall	weight_decay	fl_minoritaria	accuracy	learning_rate
0.587804	0.610599	0.602307	16	adafactor	0.858625	0.626336	0.001	0.362573	0.768577	0.000100
0.696525	0.637897	0.625644	16	schedule_free_radam	0.862385	0.664358	0.001	0.413408	0.777070	0.000030
0.676595	0.619769	0.614480	16	schedule_free_radam	0.872449	0.626676	0.001	0.367089	0.787686	0.000100
0.668119	0.630839	0.619930	32	adamw_torch	0.840108	0.680969	0.010	0.421569	0.749469	0.000005
0.649108	0.580762	0.574924	16	schedule_free_radam	0.843342	0.595990	0.010	0.318182	0.745223	0.000010
0.665294	0.619137	0.609455	16	adafactor	0.838275	0.661958	0.010	0.400000	0.745223	0.000010
0.565406	0.560112	0.565068	16	adafactor	0.872050	0.556930	0.001	0.248175	0.781316	0.000030
0.678058	0.644946	0.641356	32	schedule_free_radam	0.884664	0.649006	0.100	0.405229	0.806794	0.000030
0.652801	0.587466	0.581195	32	schedule_free_radam	0.849351	0.601028	0.010	0.325581	0.753715	0.000020
0.663490	0.604753	0.596075	32	adafactor	0.843915	0.631493	0.001	0.365591	0.749469	0.000005
0.612925	0.633193	0.628775	16	adamw_torch	0.879288	0.638471	0.100	0.387097	0.798301	0.000050
0.678058	0.644946	0.641356	32	schedule_free_radam	0.884664	0.649006	0.100	0.405229	0.806794	0.000030
0.651031	0.598193	0.590297	32	schedule_free_radam	0.844737	0.620498	0.100	0.351648	0.749469	0.000030
0.661175	0.618104	0.608249	32	schedule_free_radam	0.840268	0.657720	0.100	0.395939	0.747346	0.000030
0.672016	0.603535	0.595879	16	schedule_free_radam	0.833333	0.640428	0.001	0.373737	0.736730	0.000030

Tabla A.4: Tabla 4: Métricas y Hiperparámetros BETO *goal*.

AUC	fl-macro	precision	batch_size	optimizer	fl_mayoritaria	recall	weight_decay	fl_minoritaria	accuracy	learning_rate
0.595241	0.533753	0.546674	16	adafactor	0.768362	0.579056	0.001	0.299145	0.651805	0.000100
0.610082	0.522458	0.525220	16	adafactor	0.805333	0.535758	0.001	0.239583	0.690021	0.000005
0.621996	0.536289	0.536362	16	adafactor	0.815894	0.550071	0.010	0.256684	0.704883	0.000005
0.609946	0.517665	0.519934	16	adafactor	0.809524	0.527282	0.010	0.225806	0.694268	0.000010
0.513769	0.457373	0.421444	32	schedule_free_radam	0.914747	0.500000	0.010	0.000000	0.842887	0.000010
0.635731	0.559109	0.559109	32	adamw_torch	0.861461	0.559109	0.010	0.256757	0.766454	0.000050
0.617673	0.566147	0.562237	16	schedule_free_radam	0.829590	0.584655	0.100	0.302703	0.726115	0.000030
0.601368	0.531534	0.531375	16	schedule_free_radam	0.819974	0.541596	0.010	0.243094	0.709130	0.000050
0.624430	0.547335	0.544942	32	adamw_torch	0.831812	0.557169	0.001	0.262857	0.726115	0.000030
0.607019	0.566008	0.562460	16	schedule_free_radam	0.849807	0.572282	0.100	0.282209	0.751592	0.000100
0.601079	0.528246	0.531224	32	adamw_torch	0.802685	0.545493	0.010	0.253807	0.687898	0.000020
0.641892	0.586615	0.587615	32	adamw_torch	0.871859	0.585676	0.001	0.301370	0.783439	0.000030
0.611903	0.529562	0.542816	32	adamw_torch	0.767278	0.572299	0.001	0.291845	0.649682	0.000050
0.614252	0.458197	0.521565	32	adamw_torch	0.644550	0.540711	0.001	0.271845	0.522293	0.000050
0.595292	0.553920	0.553231	32	adamw_torch	0.813423	0.577558	0.100	0.294416	0.704883	0.000030

Tabla A.5: Tabla 5: Métricas y Hiperparámetros RoBERTa-base-bne *about*.

AUC	fl-macro	precision	batch_size	optimizer	fl_mayoritaria	recall	weight_decay	fl_minoritaria	accuracy	learning_rate
0.637007	0.542460	0.568109	32	adamw_torch	0.890229	0.539060	0.100	0.194690	0.806794	0.000005
0.661992	0.591708	0.586925	16	schedule_free_radam	0.860435	0.598849	0.001	0.322981	0.768577	0.000100
0.650725	0.558890	0.555561	16	adamw_torch	0.830486	0.573661	0.001	0.287293	0.726115	0.000010
0.652257	0.563331	0.560375	16	adamw_torch	0.851662	0.568044	0.010	0.275000	0.753715	0.000030
0.595616	0.503995	0.507154	32	adamw_torch	0.866258	0.505395	0.001	0.141732	0.768577	0.000100
0.640922	0.565338	0.561347	16	adafactor	0.844961	0.574001	0.100	0.285714	0.745223	0.000020
0.593914	0.569779	0.567308	16	adamw_torch	0.857506	0.573082	0.001	0.282051	0.762208	0.000030
0.649432	0.562817	0.559268	32	adamw_torch	0.846847	0.569763	0.001	0.278788	0.747346	0.000005
0.662128	0.591708	0.586925	16	schedule_free_radam	0.860435	0.598849	0.100	0.322981	0.768577	0.000100
0.644462	0.568351	0.565445	16	adafactor	0.820375	0.594850	0.001	0.316327	0.715499	0.000005
0.647696	0.567729	0.563358	32	schedule_free_radam	0.843061	0.578239	0.100	0.292398	0.743100	0.000050
0.661958	0.591708	0.586925	16	schedule_free_radam	0.860435	0.598849	0.010	0.322981	0.768577	0.000100
0.640990	0.566913	0.565309	16	schedule_free_radam	0.859316	0.568844	0.100	0.274510	0.764331	0.000100
0.640990	0.566913	0.565309	16	schedule_free_radam	0.859316	0.568844	0.100	0.274510	0.764331	0.000100
0.640786	0.566913	0.565309	16	schedule_free_radam	0.859316	0.568844	0.010	0.274510	0.764331	0.000100

Tabla A.6: Tabla 6: Métricas y Hiperparámetros RoBERTa-base-bne *description*.

AUC	f1-macro	precision	batch_size	optimizer	f1_mayoritaria	recall	weight_decay	f1_minoritaria	accuracy	learning_rate
0.659728	0.574495	0.569491	32	schedule_free_radam	0.833773	0.593931	0.100	0.315217	0.732484	0.000100
0.607717	0.456747	0.421277	32	schedule_free_radam	0.913495	0.498741	0.010	0.000000	0.840764	0.000030
0.649346	0.558273	0.555901	32	adafactor	0.821809	0.578358	0.010	0.294737	0.715499	0.000050
0.632327	0.557529	0.557222	32	schedule_free_radam	0.860025	0.557849	0.010	0.255034	0.764331	0.000050
0.650487	0.559660	0.556196	16	adamw_torch	0.843871	0.567244	0.001	0.275449	0.743100	0.000020
0.636139	0.575822	0.570492	32	schedule_free_radam	0.838794	0.592212	0.100	0.312849	0.738854	0.000100
0.653772	0.564415	0.570427	32	adafactor	0.788219	0.617418	0.100	0.340611	0.679406	0.000020
0.638267	0.606935	0.598275	16	adamw_torch	0.852288	0.626796	0.001	0.361582	0.760085	0.000020
0.455630	0.457373	0.421444	32	schedule_free_radam	0.914747	0.500000	0.100	0.000000	0.842887	0.000005
0.463408	0.457373	0.421444	16	adamw_torch	0.914747	0.500000	0.010	0.000000	0.842887	0.000100
0.577269	0.457373	0.421444	16	schedule_free_radam	0.914747	0.500000	0.100	0.000000	0.842887	0.000010
0.664460	0.571459	0.567339	32	adafactor	0.827128	0.594390	0.100	0.315789	0.723992	0.000020
0.636480	0.546719	0.544714	32	adafactor	0.826772	0.558888	0.100	0.266667	0.719745	0.000100
0.658639	0.563326	0.564207	32	adafactor	0.804378	0.600007	0.100	0.322275	0.696391	0.000010
0.655337	0.566972	0.573248	32	adafactor	0.787623	0.622915	0.100	0.346320	0.679406	0.000030

Tabla A.7: Tabla 7: Métricas y Hiperparámetros RoBERTa-base-bne *motivation*.

AUC	f1-macro	precision	batch_size	optimizer	f1_mayoritaria	recall	weight_decay	f1_minoritaria	accuracy	learning_rate
0.635884	0.646604	0.641137	32	adafactor	0.882952	0.653244	0.100	0.410256	0.804671	0.000050
0.656597	0.625461	0.619476	16	adafactor	0.873563	0.633433	0.010	0.377358	0.789809	0.000020
0.655491	0.634710	0.626988	32	adafactor	0.874359	0.645687	0.001	0.395062	0.791932	0.000030
0.672782	0.626334	0.615224	32	adafactor	0.844504	0.666996	0.010	0.408163	0.753715	0.000010
0.660596	0.457373	0.421444	32	schedule_free_radam	0.914747	0.500000	0.100	0.000000	0.842887	0.000005
0.675114	0.592589	0.586198	16	adamw_torch	0.853816	0.604806	0.010	0.331361	0.760085	0.000005
0.665260	0.457373	0.421444	32	schedule_free_radam	0.914747	0.500000	0.100	0.000000	0.842887	0.000020
0.662060	0.628261	0.617954	32	adafactor	0.863459	0.647866	0.100	0.393064	0.777070	0.000030
0.613299	0.610520	0.604021	16	adamw_torch	0.865212	0.620379	0.001	0.355828	0.777070	0.000020
0.631850	0.632776	0.624801	16	adamw_torch	0.872914	0.644428	0.010	0.392638	0.789809	0.000020
0.672255	0.598241	0.590455	16	adamw_torch	0.848168	0.617520	0.010	0.348315	0.753715	0.000005
0.668544	0.583649	0.586254	16	adamw_torch	0.873592	0.581438	0.010	0.293706	0.785563	0.000010
0.627732	0.615606	0.613280	16	adafactor	0.875949	0.618201	0.010	0.355263	0.791932	0.000010
0.607087	0.633243	0.633992	32	adamw_torch	0.885535	0.632514	0.010	0.380952	0.806794	0.000100
0.659371	0.457373	0.421444	32	schedule_free_radam	0.914747	0.500000	0.010	0.000000	0.842887	0.000010

Tabla A.8: Tabla 8: Métricas y Hiperparámetros RoBERTa-base-bne *goal*.

A.2. Repositorio de código utilizado

Con el objetivo de promover la transparencia y facilitar la reproducibilidad de los resultados obtenidos en este Trabajo Fin de Grado, el código desarrollado ha sido organizado y puesto a disposición en un repositorio público de GitHub. Dicho repositorio incluye:

- **Procesamiento de datos:** *notebook* para la limpieza, transformación y preparación del conjunto de datos original.
- **Entrenamiento y evaluación de modelos:** códigos para la implementación de modelos de aprendizaje automático y posterior evaluación de ensembles.
- **Interpretabilidad:** herramientas y visualizaciones que permiten comprender el comportamiento y las decisiones de los modelos entrenados.
- **Datos:** archivos CSV utilizados durante el desarrollo del proyecto.
- **Demo:** aplicación de demostración con el meta-clasificador integrado.

El repositorio se encuentra disponible en la siguiente URL:

<https://github.com/DanielDoradoMartinez/TFG>